

2Chance Web Application

Sustainable Software Engineering Project Report

Aldo Manco

23 January 2026

Contents

Chapter 1

Introduzione e Baseline del Sistema

Chapter 2

Panoramica dell'Architettura del Software

2.1 Layered Architecture Components Breakdown

The *2Chance* platform relies on a modular layered architecture that follows the traditional Model–View–Controller (MVC) paradigm. Accordingly, system responsibilities are rigorously mapped: data presentation is managed by the View (JavaServer Pages and JavaScript); input coordination is handled by the Controller; and the Model is subdivided into Services (business logic), Data Access Objects (persistence), and Beans (data structures). This strict decoupling constitutes the existing foundation upon which the project's dependability enhancements are applied.

2.1.1 The View Layer (Presentation and Interaction)

Component: `webapp` package (e.g. `index.jsp`, `login.jsp`).

The View layer constitutes the presentational logic and is responsible for rendering the user interface (UI) and capturing user inputs. It is implemented using web standard languages, including HyperText Markup Language (HTML), Cascading Style Sheets (CSS) for styling, and JavaScript (JS) for client-side interactivity. Using JavaServer Pages (JSP), it dynamically displays data provided by the Controller typically passed via request or session attributes.

Although the View initiates the data flow by transmitting user parameters via HTTP requests, it is architecturally prohibited from executing business logic or accessing the persistence layer directly.

2.1.2 The Controller Layer (Input Coordination and Routing)

Component: `controller` package (e.g. `LoginServlet`, `RegistrazioneServlet`).

The Controller functions as the centralized entry point for server-side processing. Implemented via Java Servlets, it intercepts and manages incoming HTTP requests (specifically, `GET` and `POST`) transmitted by the client. It acts as the traffic director of the application, ensuring that user actions are routed to the appropriate business logic handlers. The Controller acts strictly as an intermediary and contains no business logic.

Its operational workflow consists of three distinct phases:

1. **Parsing:** it extracts input parameters from the request object and manages the HTTP session to preserve user state across stateless interactions.

2. **Delegation:** it invokes the appropriate methods within the Service layer, passing only the necessary arguments.
3. **Routing:** upon completion of service execution, it determines the response strategy, either forwarding the request context to a JSP view for rendering or redirecting the user to a new resource to prevent form resubmission.

2.1.3 The Service Layer (Business Logic Orchestration)

Component: `services` package (e.g. `LoginService`, `AdminService`).

The Service layer constitutes the operational core of the application, encapsulating the system's business logic. It serves as an abstraction that decouples input handling (Controller) from data storage (DAO). By isolating these concerns, the architecture centralizes business rules and makes them independent of the user interface and database technology.

Implemented as standard Java classes, Service components orchestrate complex workflows and enforce domain invariants. In particular, a Service validates inputs against application-specific rules (e.g. ensuring a password meets complexity criteria or checking stock availability) before any action is taken. To preserve architectural purity, the Service layer is prohibited from executing SQL queries directly. Instead, it coordinates persistence by invoking the Data Access layer to retrieve or persist data, ensuring that logical processing remains distinct from physical data manipulation.

2.1.4 The Data Access Layer (Persistence and DAO)

Component: `model.dao` package (e.g. `UtenteDAO`, `ProdottoDAO`).

The Data Access layer implements the Data Access Object (DAO) pattern, which abstracts and encapsulates all access to the data source. This design ensures that the rest of the application remains agnostic to the concrete database implementation, enabling potential changes in the storage mechanism without impacting higher-level logic. This layer is the only authority permitted to execute SQL commands.

Using JDBC (Java Database Connectivity) and a Tomcat connection pool for efficient resource management, DAOs manage low-level communication with the database. Their primary duties include:

1. **CRUD operations:** executing Create, Read, Update, and Delete transactions.
2. **Object–Relational Mapping (ORM):** manually transforming raw relational data (e.g. `ResultSet` rows) into high-level Java objects (Model Beans) and vice versa. This translation enables the Service layer to manipulate Java objects rather than database-specific data types.

2.1.5 The Model Beans (Data Representation)

Component: `model.beans` package (e.g. `Utente`, `Prodotto`).

Model Beans represent the fundamental functional entities of the system. Structurally, they are implemented as Plain Old Java Objects (POJOs). Their primary responsibility is data encapsulation, enabling the transport of state across the View, Controller, Service, and Data Access layers.

Each Bean mirrors a specific entity in the domain model, typically corresponding to a table in the relational database. Beans maintain state through private attributes, accessed and modified exclusively through public accessor (getter) and mutator (setter) methods.

2.2 Dynamic Behavior: The Authentication Workflow

To illustrate the dynamic interaction among the architectural components, we examine the control flow of a standard user authentication (login) procedure. This sequence shows how data propagates from the user interface to the database and back, while respecting the separation of concerns defined above:

1. **Interaction and request (View layer):** the user enters credentials in `login.jsp`. Upon submission, the browser generates an HTTP POST request containing the parameters (email and password) and transmits it to the server.
2. **Interception and delegation (Controller layer):** the `LoginServlet` intercepts the request and parses the body to extract input parameters. In accordance with its coordinating role, it performs no data verification; instead, it delegates the operation by invoking `login()` on `LoginService`.
3. **Orchestration and validation (Service layer):** `LoginService` receives the raw parameters and performs preliminary validation (e.g. ensuring fields are not empty). If validation succeeds, it requests data retrieval by calling `getUserByEmailPassword()` on `UtenteDAO`.
4. **Query execution (Data Access layer):** `UtenteDAO` establishes a database connection and constructs and executes a secure `SELECT` query. This is the only point in the flow where the application communicates directly with persistence storage.
5. **Object mapping (Data Access layer):** from the returned `ResultSet`, the DAO performs object-relational mapping: it instantiates a new `Utente` Bean, populates its fields with database values, and returns the object to the Service layer.
6. **State management and response (Controller layer):** the Service returns the populated `Utente` Bean to the Controller. The Controller then (i) persists state by storing the Bean into the HTTP session, effectively logging the user in, and (ii) performs navigation by sending a redirect response instructing the client to load `dashboard.jsp`, where the user's name is rendered.

Chapter 3

Sostenibilità Energetica del Back-End

Il livello Back-End di 2Chance, che orchestra la logica di business, l'interazione con il database relazionale e la manipolazione delle stringhe di interfaccia, rappresenta il motore computazionale primario del sistema e, di conseguenza, il principale vettore di consumo energetico. L'obiettivo di questa fase è triplice: misurare l'assorbimento dinamico di potenza durante l'esecuzione, identificare tramite analisi statica i pattern di codice ecologicamente inefficienti e rifattorizzare le componenti per minimizzare lo spreco di risorse.

3.1 Misurazione Dinamica dell'Energia tramite EnergiBridge e JMeter

Per acquisire metriche empiriche sul consumo energetico a livello hardware, il framework metodologico prevede l'integrazione di EnergiBridge. EnergiBridge è uno strumento di profilazione *cross-platform* all'avanguardia, concepito specificamente per consentire agli ingegneri che utilizzano diversi ecosistemi operativi di essere consapevoli del consumo energetico delle loro applicazioni. La sua caratteristica fondamentale risiede nella capacità di interfacciarsi a basso livello con i sensori hardware di sistema, supportando architetture x86, AMD, Apple ARM (M1/M2/M3),... Questa compatibilità è vitale per il progetto 2Chance, le cui misurazioni verranno condotte su un'architettura AMD Ryzen AI 9.

3.1.1 Configurazione dell'Ambiente di Profilazione

Per l'installazione e la configurazione di EnergiBridge in ambiente Windows, è stata predisposta la directory di sistema `C:\Program Files\EnergiBridge`. Successivamente, è stata prelevata l'ultima *release* stabile (0.0.7) dal repository ufficiale GitHub ([tdurieux/EnergiBridge](https://github.com/tdurieux/EnergiBridge)), congiuntamente a `LibreHardwareMonitor`, un plugin indispensabile per abilitare l'accesso ai registri della CPU su piattaforma Windows.

3.1.2 Strategia di Monitoraggio e Workload Deterministico

EnergiBridge opera prevalentemente da riga di comando, campionando i consumi energetici a intervalli di tempo predefiniti durante l'esecuzione di un processo *target* e restituendo i dati aggregati in formato CSV per successive analisi. Poiché EnergiBridge misura l'impatto di un processo in esecuzione, è strettamente necessario generare un carico di lavoro riproducibile, deterministico e continuativo.

A tal fine, Apache JMeter viene impiegato per orchestrare simulazioni massive di traffico utente. La strategia combinata prevede lo sviluppo di script JMeter (`.jmx`) configurati per

l'esecuzione in modalità *non-GUI*, simulando il comportamento simultaneo di molteplici utenti. Gli script includono vari scenari di utilizzo:

- **Load Testing:** per valutare il consumo energetico con un carico previsto che l'infrastruttura deve essere in grado di gestire.
- **Soak Testing:** per osservare l'accumulo di consumo energetico e i *leak* di memoria su un periodo prolungato con un carico lieve previsto che l'infrastruttura deve essere in grado di gestire, un aspetto essenziale in un ambiente basato su Java (JVM).
- **Spike Testing:** per misurare il picco di assorbimento (misurato in Watt) durante ondate repentine di traffico con un carico superiore a quello che l'infrastruttura è in grado di supportare.

Durante l'esecuzione di questi scenari tramite JMeter, EnergiBridge registra il comportamento energetico della macchina. Siccome l'obiettivo è fare un refactoring orientato ad una maggiore sostenibilità del software, verranno confrontati i dati raccolti dalle analisi del *branch* di *baseline*, che rappresenta il sistema prima del refactoring, e il *branch sse-improvements* che rappresenta il sistema dopo il refactoring orientato alla sostenibilità del software. Questo confronto permetterà di quantificare oggettivamente l'impatto del refactoring orientato alla sostenibilità.

Le metriche chiave estratte ed elaborate dai file CSV di EnergiBridge sono strutturate nella Tabella ??.

3.1.3 Progettazione degli Scenari di Test JMeter

La strategia di simulazione si focalizza sul sollecitare esclusivamente le *servlet* utilizzate dagli utenti. Questo approccio garantisce che la valutazione delle inefficienze e dei *bottleneck* architetturali avvenga su carichi di lavoro realistici, permettendo di osservare se il sistema presenta degradazioni non lineari (es. esplosioni combinatorie di calcolo o query inefficienti) che peggiorano severamente la sostenibilità sotto stress.

Sono state deliberatamente omesse le *servlet* dedicate all'amministrazione del sistema (`/AdminServlet/` e `/InitServlet`), in quanto accessibili solo da un'utenza privilegiata molto ristretta e non soggette a traffico massivo o continuativo.

Di seguito, le *servlet* sottoposte a stress test, categorizzate per area funzionale:

- **Navigazione e Catalogo (Lettura intensiva):**
 - `/landingpage`: Carica la homepage e la vetrina iniziale.
 - `/ProdottiPerCategoriaServlet`: Filtra e mostra i prodotti in base alla categoria.
 - `/ProdottoServlet`: Mostra la pagina di dettaglio di un singolo articolo.
 - `/RicercaServlet`: Gestisce la barra di ricerca globale del catalogo.
- **Autenticazione e Profilo (Transazionali/Lettura):**
 - `/LoginServlet`: Gestisce il login degli utenti.
 - `/RegistrazioneServlet/*`: Gestisce la creazione degli account per i nuovi utenti.
 - `/logoutServlet`: Invalida la sessione utente in corso.
 - `/UtenteServlet`: Mostra la pagina del profilo utente.
- **E-commerce: Carrello e Ordini (Transazionali):**

Metrica di Sostenibilità	Descrizione Analitica e Impatto Ecologico	Unità di Misura
Energia CPU Package	Assorbimento energetico totale del processore durante l'esecuzione (inclusi core, cache L2/L3 e controller integrati). Indica il “costo netto” dell'operazione.	Joules (J)
Potenza Media CPU Package	Il tasso medio di consumo energetico nel tempo (Energia/Tempo). Un abbassamento indica un codice che permette al processore di operare in stati di risparmio energetico (C-states).	Watt (W)
Energia PPO (Core)	Consumo limitato ai soli core elaborativi, escludendo la porzione “uncore” (es. GPU integrata). È l'indicatore più diretto dell'efficienza algoritmica.	Joules (J)
CPU Usage (Media e Picco)	La percentuale di cicli di clock effettivamente impegnati. Picchi anomali o un uso medio costantemente elevato indicano inefficienze asintotiche o attese attive (spin-locks).	Percentuale (%)
Frequenza Media CPU	La velocità di clock mantenuta dal processore. Algoritmi ottimizzati permettono l'esecuzione a frequenze inferiori, riducendo esponenzialmente la tensione applicata e il calore dissipato.	MHz
Memoria RAM Usata (Media e Picco)	L'impronta allocata nel sistema. In un ambiente basato su Java Virtual Machine (JVM) come 2Chance, il contenimento della memoria è cruciale per evitare cicli dispendiosi di Garbage Collection.	MB/GB

Table 3.1: Metriche di sostenibilità hardware campionate.

- `/AggiungiCarrelloServlet`: Aggiunge un nuovo articolo al carrello dell'utente autenticato.
- `/EditCarrello`: Modifica le quantità degli articoli nel carrello.
- `/CheckoutServlet`: Finalizza l'acquisto ed pubblica l'ordine sul database.

- **Social e Funzionalità Aggiuntive:**

- `/WishlistServlet`: Gestisce la lista dei desideri dell'utente autenticato.
- `/ConfrontaServlet`: Permette il confronto tra le caratteristiche tecniche di vari prodotti.
- `/RecensioniServlet/*`: Gestisce l'inserimento e la lettura delle recensioni.

Simulazione del Flusso Utente (User Journey)

Per orchestrare simulazioni di *load*, *spike* e *soak testing* verosimili, è stato modellato un tipico percorso d'uso, o *User Journey*, in grado di sollecitare orizzontalmente l'applicazione. Il flusso utente simulato segue una catena causale sequenziale di interazioni *client-server*:

1. L'utente si iscrive alla *web application* (`/RegistrazioneServlet/*`).

2. Approda sulla vetrina principale del portale (/landingpage).
3. Accede alla propria area riservata per gestire l'account (/UtenteServlet).
4. Esplora il catalogo visualizzando i prodotti per categoria (/ProdottiPerCategoriaServlet).
5. Analizza i dettagli di uno specifico prodotto di interesse (/ProdottoServlet).
6. Inserisce il prodotto nella propria lista dei desideri (/WishlistServlet).
7. Effettua una ricerca mirata per trovare un altro prodotto specifico (/RicercaServlet).
8. Accede alla pagina del nuovo prodotto cercato (/ProdottoServlet).
9. Esegue un confronto tecnico tra il nuovo prodotto e quello salvato in precedenza nella *wishlist* (/ConfrontaServlet).
10. Aggiunge al carrello il prodotto ritenuto più adeguato (/AggiungiCarrelloServlet).
11. Modifica il carrello per incrementare le quantità di acquisto (/EditCarrello).
12. Finalizza la transazione d'acquisto (/CheckoutServlet).
13. Accede alla sezione recensioni e pubblica un feedback sull'articolo acquistato (/RecensioniServlet/*).
14. Termina la sessione per uscire dal proprio account (/logoutServlet).
15. L'utente si riautentica (/LoginServlet).
16. Approda nuovamente sulla vetrina principale del portale (/landingpage).

Analisi del Collo di Bottiglia sotto Carico Elevato

L'applicazione mostra una stabilità ottimale con carichi moderati (fino a 32 utenti concorrenti), mantenendo i tempi di risposta medi al di sotto dello SLA prestabilito di 1200ms. Tuttavia, all'aumentare del carico oltre tale soglia, si innesca un degrado non lineare dei tempi di risposta a causa del queuing infrastrutturale.

Questo comportamento è riconducibile alla saturazione del pool di connessioni del database, gestito dalla classe `ConPool` con un limite massimo di 50 connessioni attive (`setMaxActive(50)`). Dal punto di vista teorico, ci si potrebbe attendere che il sistema generi errori non appena il numero di utenti concorrenti supera 50. Tuttavia, l'analisi empirica dello Step-Up mostra che il tasso di errore rimane allo 0.00% fino a 80 utenti concorrenti. Questo fenomeno si spiega con il fatto che i thread di JMeter non richiedono una connessione al database nello stesso preciso istante temporale, a causa dei tempi di elaborazione della CPU, della latenza di rete e dei passaggi dello User Journey che non effettuano interrogazioni persistenti.

Ciononostante, quando il carico raggiunge gli 86 utenti concorrenti, la frequenza di richieste simultanee supera stabilmente la capacità del pool (50 connessioni). Di conseguenza, le richieste in eccesso rimangono bloccate in coda in attesa di una connessione libera per un tempo superiore al limite di `maxWait` (10.000 ms), innescando timeout fisiologici (tasso di errore dell'8.05% a 86 utenti, che sale al 58.62% a 92 utenti). Questo collo di bottiglia costringe la CPU del server a uno stato di attesa attiva (*active waiting*) e a ripetute ritrasmissioni, degradando gravemente sia la reattività del servizio che l'efficienza energetica dell'infrastruttura.

3.1.4 Definizione delle Soglie di Accettabilità (SLAs) e Dimensionamento dei Test

Esperimento di Stress Preventivo: Determinazione del Punto di Rottura (Capacità Nominale C)

Prima di procedere all'esecuzione formale degli scenari di stress, è stato eseguito un esperimento preventivo di *Step-Up Testing* volto a individuare la capacità massima reale del sistema (*breaking point*) sul laptop di riferimento. Questo scenario incrementale è stato configurato per aumentare costantemente il carico in modo lineare, partendo da 1 utente fino a raggiungere 120 utenti concorrenti nell'arco di 10 minuti (con un incremento costante), monitorando l'andamento dei tempi di risposta in JMeter e lo stato dei container Docker.

L'analisi dei risultati reali dello Step-Up ha evidenziato il seguente comportamento dinamico:

- **Fase di Esecuzione Nominale (1-32 utenti):** La curva dei tempi di risposta si mantiene entro la soglia di accettabilità (< 1200 ms). Il tempo medio di risposta è di appena 269ms a 8 utenti, salendo gradualmente a 1145ms a 32 utenti concorrenti, con un tasso di errore pari allo 0.00%. Il throughput cresce regolarmente fino a un picco di 33.4 richieste al secondo a 20 utenti, per poi assestarsi intorno alle 24.5 richieste al secondo a 32 utenti.
- **Punto di Saturazione e Degradazione (38-80 utenti):** Oltre i 32 utenti, il sistema viola lo SLA di latenza prestabilito. A 38 utenti la latenza media sale a 1480ms (throughput 23.8/s). All'aumentare degli utenti la latenza sale progressivamente, raggiungendo i 2553ms a 80 utenti concorrenti (throughput di 30.1/s), sebbene il tasso di errore rimanga ancora dello 0.00%.
- **Punto di Rottura (> 80 utenti):** A 86 utenti concorrenti si registrano le prime anomalie con un tasso di errore dell'8.05% (54 fallimenti) e una latenza media di 3347ms. Oltre questa soglia, al raggiungimento dei 92 utenti concorrenti, il tasso di errore sale drasticamente al 58.62% con una latenza media di 7008ms, fino a concludere il test a 120 utenti con un tasso di errore del 70.60% e oltre 8.4 secondi di latenza media, a causa della saturazione delle connessioni nel pool (`maxActive` = 50) e del timeout in coda.

Sulla base di questo esperimento empirico, la capacità nominale massima gestibile in stabilità (C) rispettando lo SLA di latenza per la configurazione attuale è stata impostata a **32 utenti concorrenti**. Questo valore garantisce che il sistema risponda in tempi accettabili prima che inizi il degrado indotto dal queuing nel connection pool.

Di conseguenza, per ciascuno scenario sono state stabilite le seguenti configurazioni e soglie operative:

1. Scenario di Load Testing (Carico di Picco Sostenibile)

L'obiettivo è testare l'applicazione al limite superiore della sua capacità nominale.

- **Configurazione del Carico:** 32 utenti concorrenti (`threads` = 32), ramp-up di 32 secondi, durata di 10 minuti (`duration` = 600 secondi).
- **Soglia di Errore (Error Rate SLA):** $< 1.0\%$.
- **Tempo di Risposta (Latency SLA):** Tempo medio di risposta < 1200 ms; 95-esimo percentile delle richieste < 2200 ms.
- **Consumo Energetico (Soglia Green):** Potenza media della CPU Package < 25 Watt.

2. Scenario di Soak Testing (Resistenza e Memory Leak)

Lo scopo è monitorare il comportamento a lungo termine sotto un carico moderato ma costante (pari al 50% della capacità nominale C).

- **Configurazione del Carico:** 16 utenti concorrenti (`threads` = 16), ramp-up di 16 secondi, durata di 2 ore (`duration` = 7200 secondi).
- **Soglia di Errore (Error Rate SLA):** $< 0.1\%$.
- **Tempo di Risposta (Latency SLA):** Tempo medio < 600 ms, mantenuto stabile ed entro una deviazione standard del $\pm 10\%$ per l'intera durata del test.
- **Consumo Energetico (Soglia Green):** Consumo medio CPU Package < 18 Watt.

3. Scenario di Spike Testing (Tolleranza ai Picchi Improvvisi)

Si valuta la robustezza strutturale a fronte di un sovraccarico repentino superiore alla capacità nominale.

- **Configurazione del Carico:** 48 utenti concorrenti (`threads` = 48), ramp-up in 5 secondi (`rampup` = 5), durata di 2 minuti (`duration` = 120 secondi).
- **Soglia di Errore (Error Rate SLA):** $< 25\%$. Poiché il connection pool è limitato a 50 connessioni attive, le richieste eccedenti andranno in coda e subiranno timeout fisiologici a causa del superamento di `maxWait`.
- **Tempo di Risposta (Latency SLA):** Media globale < 12000 ms (inclusi i 10 secondi di coda nel pool).
- **Consumo Energetico (Soglia Green):** Picco di potenza CPU Package fino a 54 Watt.

3.1.5 Configurazione Architettuale dei Test JMeter

Sebbene Apache JMeter offra una ricca interfaccia grafica per la progettazione e la configurazione dei test, la sua persistenza sottostante si basa su file strutturati che fungono da veri e propri copioni operativi. Trattare questi file da una prospettiva architettuale permette di comprendere a fondo le dinamiche del carico generato per stressare il sistema.

L'architettura concettuale dei test è suddivisa in specifici strati di orchestrazione:

Astrazione del Contesto di Rete (Test Plan) Il livello fondamentale del test definisce la configurazione globale della simulazione. Un paradigma elegante adottato è l'uso intensivo di variabili globali per le coordinate di rete (come indirizzo del server, porta e contesto dell'applicazione). Questa astrazione disaccoppia il test dallo specifico ambiente di esecuzione, garantendo un'elevata flessibilità e rendendo l'adattamento verso ambienti di *staging* o produzione un'operazione immediata e priva di attriti strutturali.

Orchestrazione della Concorrenza (Thread Group) La simulazione del traffico è governata da un gestore della concorrenza che istanzia gli utenti virtuali. In questo scenario, il test è progettato per generare un numero predefinito di utenti concorrenti. Per evitare che tali utenti attacchino simultaneamente il sistema generando picchi irrealistici e non rappresentativi del traffico organico (il cosiddetto fenomeno del *thundering herd*), il sistema introduce un periodo di *ramp-up*. Questo meccanismo distribuisce le inizializzazioni lungo una finestra temporale,

portando gradualmente il carico fino al picco desiderato, per poi mantenerlo costante lungo un *plateau* temporale (fase di stress ininterrotto) idoneo a consentire una campionatura energetica statisticamente rilevante.

Mantenimento dello Stato e dell'Isolamento (Session Management) Affinché un utente virtuale possa compiere un percorso navigazionale coerente, è imperativo emulare il comportamento di un *browser* reale nel mantenimento dello stato. A tale scopo, la configurazione intercetta e conserva i *cookie* di sessione generati dal server (es. `JSESSIONID`). Un aspetto cruciale di questa configurazione è la garanzia di isolamento tra le iterazioni: ogni qualvolta un utente virtuale ricomincia il suo ciclo, il contesto di sessione viene ripulito, assicurando che le singole esecuzioni non subiscano interferenze dovute a *cache* o *cookie* persistenti da cicli precedenti.

Campionamento delle Interazioni (HTTP Samplers) Il nucleo operativo della simulazione è costituito dai campionatori, che replicano fedelmente le richieste HTTP previste dallo *User Journey*. Si distinguono due tipologie primarie di interazioni:

- **Lettura Informativa (GET):** Operazioni idempotenti, tipiche dell'esplorazione del catalogo o della visualizzazione dei profili. Queste richieste caricano la rete e le risorse elaborative per l'interrogazione al database, ma hanno un impatto circoscritto sulle scritture persistenti.
- **Mutazione di Stato (POST):** Operazioni transazionali (es. registrazione utente, check-out). Un dettaglio ingegneristico essenziale in queste fasi è l'introduzione dell'entropia. Ad esempio, per evitare eccezioni di unicità sul database quando molteplici utenti tentano la registrazione contemporaneamente, il sistema inietta stringhe pseudo-casuali per generare dinamicamente indirizzi email e parametri univoci, scongiurando collisioni deterministiche.

L'esecuzione sequenziale e parallela di questi blocchi costringe l'applicazione a reagire a un flusso di carico realistico. La tracciatura sincronizzata di tali eventi, accoppiata al profilatore energetico, fornisce il *dataset* primario alla base dell'analisi di sostenibilità del sistema.

3.1.6 Automazione, Isolamento e Sincronizzazione dei Test (backend_energy_co

Al fine di garantire l'affidabilità scientifica, la riproducibilità e l'assenza di bias sperimentali durante le sessioni di misurazione dinamica del consumo energetico, è stato sviluppato uno script di orchestrazione in PowerShell (`backend_energy_consumption_analysis.ps1`). Dal punto di vista della metodologia di sperimentazione empirica, le scelte architetturali adottate in questo script rispondono a precisi requisiti teorici:

Isolamento dell'Ambiente e Prevenzione del Software Aging

Durante l'esecuzione di test di carico su macchine virtuali o containerizzate in Java (JVM), le prestazioni del sistema possono degradare progressivamente a causa del fenomeno noto come *software aging* (invecchiamento del software). Questo degrado è indotto da:

- **Frammentazione della Memoria ed Accumulo di Garbage Collection (GC):** Test consecutivi eseguiti sulla stessa istanza del server Tomcat (container `webapp`) causano un progressivo accumulo di oggetti nella memoria Heap. Questo costringe la JVM a invocare con frequenza crescente cicli di *Full Garbage Collection*, caratterizzati da pause di tipo *stop-the-world*. Tali pause introducono latenze artificiali e aumentano l'overhead

computazionale della CPU, contaminando le metriche di consumo energetico rilevate da EnergiBridge.

- **Perdite di Connessioni al Database (Connection Leaks):** Sebbene il connection pool sia sintonizzato per rilasciare connessioni inattive, eventuali anomalie applicative o eccezioni non gestite che omettono la chiusura formale delle connessioni JDBC (`Connection`, `Statement`, `ResultSet`) possono causare la saturazione del pool di `ConPool` (limitato a un massimo di 50 connessioni attive). Quando la capacità massima viene raggiunta, le richieste successive subiscono un ritardo di accodamento pari a `maxWait` (10.000 ms), sfociando infine in timeout.

Per mitigare teoricamente questi fattori di distorsione, lo script esegue la sequenza:

```
docker-compose down
docker-compose up -d
docker-compose restart webapp mysql-db
```

Questo flusso garantisce l'azzeramento dello stato della memoria Heap della JVM e la rimozione di qualsiasi socket o transazione pendente sul database MySQL prima di ciascun esperimento.

Warm-Up del Server e Risoluzione dei Transitori

La fase di bootstrap di un'applicazione web in ambiente Java comporta un carico computazionale transitorio molto intenso dovuto a operazioni di classloading, parsing del descrittore di deployment, inizializzazione dei framework interni e pre-popolazione delle connessioni del pool (configurato a una dimensione iniziale di 5 connessioni). Inviare traffico in questa fase comporterebbe un tasso di errore fittizio ed un picco di consumo energetico non correlato alle normali operazioni di business. L'integrazione di una istruzione di attesa (`Start-Sleep -Seconds 10`) permette al server Tomcat di completare la fase di warm-up e di stabilizzarsi in uno stato di *idle* prima che inizi la profilazione.

Sincronizzazione della Profilazione ed Esecuzione Headless

Dal punto di vista dell'analisi del consumo energetico, la telemetria deve focalizzarsi esclusivamente sulla finestra temporale di esecuzione del carico di lavoro prestabilito. Per questo motivo, EnergiBridge viene invocato come processo wrapper del comando JMeter:

```
& "energibridge.exe" -o ... — docker-compose exec jmeter jmeter -n ...
```

In questo modo, la raccolta dei dati sul consumo di potenza (in Watt) e sull'uso delle risorse hardware coincide perfettamente con la durata del test pianificato. Inoltre, l'invocazione di JMeter all'interno del container avviene rigorosamente in modalità *non-GUI* (opzione `-n`). L'avvio dell'interfaccia grafica su un computer host comporterebbe infatti un consumo energetico addizionale e asimmetrico dovuto al rendering Swing della GUI di JMeter, contaminando pesantemente le misurazioni di baseline.

Separazione delle Preoccupazioni nella Telemetria

Lo script organizza in modo distinto la persistenza dei dati generati, incanalando le metriche prestazionali (file log prestazionali `.jtl` contenenti latenze di rete, tempi di servizio e codici di stato HTTP) all'interno della directory `jmeter_logs`, e la telemetria energetica (file `.csv` registrati da EnergiBridge) nella cartella `energibridge_logs`. Questa separazione logica semplifica l'elaborazione dei dati e garantisce l'isolamento tra l'analisi prestazionale classica (rispetto degli SLA) e la quantificazione del consumo ecologico (sostenibilità energetica).

Orchestrazione Automatizzata e Analisi End-to-End

Per ottimizzare il flusso di lavoro ed evitare disallineamenti temporali o errori umani nella fase di post-elaborazione dei dati energetici, l'architettura integra l'intera pipeline sperimentale in un unico script di orchestrazione automatizzato. Invece di richiedere l'avvio manuale di script separati per ciascuna fase (configurazione dell'ambiente, esecuzione del carico, cattura della telemetria ed estrazione dei risultati), il file `backend_energy_consumption_analysis.ps1` esegue sequenzialmente e in modo autonomo le seguenti operazioni:

1. **Setup dell'Ambiente:** Esegue il ciclo di teardown e startup di Docker per azzerare lo stato della JVM e delle connessioni JDBC.
2. **Profilazione Dinamica:** Esegue la sessione di carico JMeter non-GUI con telemetria EnergiBridge attiva.
3. **Post-Elaborazione Automatica:** Al termine del test di carico, lo script invoca automaticamente il modulo Python `energibridge_metrics_extractor.py` passandogli il tracciato CSV appena generato.
4. **Persistenza e Logging Sincronizzato:** L'output dell'estrazione energetica viene stampato sulla console e contemporaneamente persistito tramite il comando `Tee-Object` all'interno della cartella `extractor_logs`. Il file di log energetico (es. `log_load_extractor_2026_05_21_20_47`) eredita la medesima marcatura temporale (*timestamp*) generata all'inizio del test.

Questo approccio end-to-end garantisce la riproducibilità scientifica dell'esperimento: ogni sessione di test produce in modo atomico e autocontenuto un set completo di file di output correlati tra loro dallo stesso identico timestamp (file `.jtl` prestazionale, file `.csv` di telemetria grezza, file `.log` di JMeter e file `.log` dell'estrattore energetico). In questo modo, l'intero ciclo di vita del test e dell'analisi energetica viene gestito da un singolo punto d'ingresso, garantendo coerenza metodologica e riducendo a zero l'overhead manuale.

3.1.7 Analisi Energetica Post-Elaborazione ed Estrazione delle Metriche (`energibridge_metrics_extractor.py`)

Al fine di tradurre i dati grezzi raccolti da EnergiBridge e Apache JMeter in informazioni utili per l'ingegneria del software sostenibile, è stata sviluppata una pipeline di post-elaborazione in Python (`energibridge_metrics_extractor.py`). Lo script si occupa di correlare le misurazioni fisiche del consumo energetico dell'hardware (file CSV di EnergiBridge) con le richieste HTTP intercettate dall'applicazione (file JTL di JMeter), consentendo di valutare sia l'efficienza energetica globale che quella specifica di ciascuna *servlet*.

Allineamento Temporale e Correlazione dei Log

L'orchestrazione dei test descritta in precedenza genera tracciati con marcatura temporale sincronizzata (*timestamp*). Lo script Python riceve in input il percorso del file CSV di EnergiBridge, estrae il timestamp dal nome del file (formato `YYYY-MM-DD_hh-mm-ss`) e ricerca nella directory `jmeter_logs` il file JTL corrispondente.

Qualora alcune righe del file CSV presentino marcature temporali mancanti o incomplete, lo script esegue un allineamento predittivo interpolando i record partendo dal timestamp iniziale (del JTL o del primo record valido di EnergiBridge) e incrementandolo del tasso di campionamento nominale (200 ms).

Filtrazione del Rumore dei Sensori RAPL

I sensori fisici basati su Intel/AMD RAPL (*Running Average Power Limit*) o registri equivalenti possono essere soggetti a rumore hardware, caratterizzato da oscillazioni fittizie e repentine nei valori assoluti di energia (ad esempio, salti istantanei da decine a migliaia di Joule). Se calcolato tramite semplice differenza tra l'ultimo e il primo valore registrato ($E_{end} - E_{start}$), il consumo totale risulterebbe pesantemente sovrastimato o falsato.

Per risolvere questo problema, lo script implementa un filtro di soglia basato sulla potenza fisica teorica massima del processore. Per ogni coppia di record consecutivi $k - 1$ e k , viene calcolato l'intervallo temporale $\Delta t = (T_k - T_{k-1})/1000$ (in secondi) e la differenza di energia letta dai sensori $\Delta E = E_k - E_{k-1}$. La differenza viene considerata valida solo se rispetta le seguenti restrizioni fisiche:

- **Package CPU Energy:** $0 \leq \Delta E_{CPU} \leq 250.0 \times \Delta t$. Una variazione superiore a 250 W per frazione di secondo viene scartata (imposta a 0) in quanto fisicamente impossibile per la CPU di riferimento.
- **Core CPU Energy (PPO):** $0 \leq \Delta E_{Core} \leq 150.0 \times \Delta t$. Variazioni superiori a 150 W sui singoli core elaborativi vengono analogamente filtrate.

Questo approccio consente di accumulare solo i contributi reali di consumo energetico, neutralizzando i picchi spuri e rendendo le misurazioni stabili e confrontabili.

Calcolo delle Metriche Globali di Sistema

Le metriche di sostenibilità globali sono calcolate tramite sommatoria dei delta filtrati per l'intero set di record M :

- **Durata Totale dell'Analisi (T_{global}):**

$$T_{global} = \sum_{k=1}^M \Delta t_k \quad (3.1)$$

- **Energia Totale CPU Package (E_{CPU}):**

$$E_{CPU} = \sum_{k=1}^M \Delta E_{CPU,k} \quad (3.2)$$

- **Energia Totale PPO/Core (E_{Core}):**

$$E_{Core} = \sum_{k=1}^M \Delta E_{Core,k} \quad (3.3)$$

- **Potenza Media CPU Package (P_{avg}):**

$$P_{avg} = \frac{E_{CPU}}{T_{global}} \quad (3.4)$$

Le metriche di utilizzo CPU, frequenza e memoria RAM usata vengono aggregate calcolandone il valore medio e il picco massimo registrato durante la sessione.

Calcolo delle Metriche per Singola Servlet (Algoritmo Sweep-Line)

Assegnare il consumo hardware complessivo alle singole componenti software rappresenta una sfida teorica, in quanto molteplici richieste HTTP possono essere elaborate in parallelo dai thread di Tomcat. Lo script risolve questo problema applicando un algoritmo di attribuzione proporzionale a grana fine basato su time-slice, ottimizzato tramite una tecnica *sweep-line*.

Per ogni time-slice delimitato da due campionamenti di EnergiBridge $[T_{k-1}, T_k]$:

1. Viene identificato l'insieme delle richieste HTTP attive in quell'intervallo temporale. Una richiesta i , definita dal suo intervallo $[start_i, end_i]$, è considerata attiva nella slice se:

$$start_i \leq T_k \quad \wedge \quad end_i \geq T_{k-1} \quad (3.5)$$

2. Si calcola la concorrenza locale N_{active} , ovvero il numero di richieste simultanee che condividono la risorsa elaborativa nella slice.
3. Se $N_{active} > 0$, l'energia consumata dalla CPU durante la slice ($\Delta E_{CPU,k}$ e $\Delta E_{Core,k}$) viene suddivisa equamente tra le richieste attive (*Fair Sharing Rule*):

$$Share_{CPU} = \frac{\Delta E_{CPU,k}}{N_{active}}, \quad Share_{Core} = \frac{\Delta E_{Core,k}}{N_{active}} \quad (3.6)$$

L'energia calcolata viene accreditata alle rispettive *servlet* di appartenenza.

4. Per ogni servlet attiva nella slice, vengono memorizzati i campioni istantanei di utilizzo della CPU, frequenza di clock e allocazione della memoria RAM, e viene incrementata la durata cumulativa di attività del servlet.

L'algoritmo sweep-line mantiene un pool attivo di richieste scorrendo linearmente i record di EnergiBridge. Avendo ordinato preventivamente le richieste JMeter per tempo di inizio, la complessità computazionale è lineare $O(N + M)$ (dove N è il numero totale di richieste HTTP e M il numero di campioni energetici), evitando scansioni quadratiche ridondanti.

Al termine del ciclo, per ogni servlet s vengono estratte le seguenti metriche di performance e sostenibilità:

- **Requests:** Numero di invocazioni totali della servlet.
- **Avg Resp Time:** Tempo medio di risposta calcolato direttamente dal log prestazionale JTL.
- **CPU/Core Energy:** Somma degli share energetici attribuiti alla servlet nei diversi time-slice.
- **Avg Power:** Calcolata dividendo l'energia CPU attribuita per il tempo totale in cui la servlet è stata attiva.
- **Avg/Peak CPU e RAM:** Statistiche descrittive (medie e picchi) calcolate unicamente sulle slice in cui la servlet era in esecuzione.

Esportazione Multicanale

Infine, lo script genera due report separati in formato testuale: un report globale focalizzato sull'infrastruttura energetica complessiva e un report di unità strutturato in tabella per l'analisi comparativa delle singole servlet. Per garantire che i log fisici siano facilmente analizzabili da tool di monitoraggio automatici, lo script rimuove tutte le sequenze di escape ANSI (utilizzate per colorare l'output su console) prima di scrivere i file nelle directory `performance_system_testing_logs` e `performance_unit_testing_logs`.

3.1.8 Classificazione Energetica dei Componenti e Target di Ottimizzazione

Sulla base di una rigorosa analisi empirica dei dataset di test di Load, Soak e Spike, è possibile categorizzare i componenti del back-end dell'applicazione in tre livelli (*tier*) distinti: **Target Definiti per JMH + EnergiBridge**, **Target Secondari/Condizionali** e **Componenti da Saltare** (*Components to Skip*). Questa classificazione consente di ottimizzare il budget e lo sforzo di sviluppo, evitando di dedicare tempo alla scrittura di microbenchmark per porzioni di codice che non produrrebbero una riduzione significativa del consumo energetico ($\Delta\%$).

Target Definiti: Componenti Prioritari (MUST Work On)

Questi componenti rappresentano le priorità assolute e critiche dell'attività di ottimizzazione. Le relative metriche evidenziano una sollecitazione hardware massiva e costante in tutti e tre i profili di test operativi. Essi devono essere necessariamente isolati in una pipeline combinata JMH + EnergiBridge per tracciare il loro consumo di baseline in Joule per operazione (*J/op*).

1. Il Sottosistema di Gestione delle Recensioni (Path RecensioniServlet)

- **Evidenza Empirica:** Nel test di Load, il sottosistema consuma ben 2273,44 J. Nel test di Soak, drena un quantitativo massivo di energia pari a 22.373,84 J, portando l'utilizzo della CPU al picco limite del 100% e l'allocazione di RAM fino a 23,01 GB. Durante il test di Spike, i tempi di risposta crollano drasticamente fino a 43,22 secondi, mantenendo un assorbimento di potenza medio di 11,41 W.
- **Cosa estrarre per JMH:** I metodi lato back-end per il recupero e l'ordinamento all'interno dei layer Service o DAO (es. `RecensioneDAO.getRecensioniByProdotto(...)` o eventuali routine intermedie di analisi testuale o sentiment analysis).
- **Target Creedengo:** Ricerca di inefficienze legate alla mutabilità delle stringhe e al sovraccarico da concatenazione (es. l'uso dell'operatore `+=` all'interno di un ciclo per generare dinamicamente le stringhe HTML delle recensioni) e alle varianti di ciclo invarianti (regola GC169). La rimozione di queste inefficienze ridurrà l'allocazione di picco della RAM (pari a 23,01 GB) e arresterà i cicli di Garbage Collection ad alto dispendio energetico che saturano la CPU.

2. Il Sottosistema del Catalogo Prodotti e Query (Path ProdottoServlet)

- **Evidenza Empirica:** Rappresenta il principale consumatore energetico del sistema. Registra il consumo energetico assoluto più elevato sia nel test di Load standard (3731,77 J) sia nel test di Soak prolungato (36.089,51 J), mantenendo costantemente un picco di occupazione della memoria RAM di 23,00 GB.
- **Cosa estrarre per JMH:** La logica di Object-Relational Mapping (ORM) in cui i set di risultati del database (`ResultSet`) vengono convertiti in POJO (Model Beans), come `ProdottoDAO.mapResultSetToPojo(...)`.
- **Target Creedengo:** Verifica delle violazioni della regola GC169 (invocazione di `.size()` o altre condizioni invarianti all'interno di cicli che elaborano tabelle di prodotti) ed eliminazione delle query SQL eseguite all'interno di cicli (un enorme spreco energetico che costringe i thread di Tomcat a rimanere attivi in attesa di I/O).

Target Secondari: Priorità Media (High Priority If Time Permits)

Questi componenti mostrano un elevato utilizzo di risorse, ma i loro pattern energetici rispecchiano da vicino quelli dei target primari, suggerendo che probabilmente condividono la stessa architettura sottostante o gli stessi metodi helper.

1. I Motori di Confronto e Gestione del Carrello (Path ConfrontaServlet & AggiungiCarrelloServlet)

- **Evidenza Empirica:** Entrambi i componenti consumano circa 2200 J nei test di Load e 22.000 J in quelli di Soak, registrando lo stesso picco limite di RAM a 23,01 GB. Durante improvvisi picchi di traffico (Spike), portano l'utilizzo della CPU rispettivamente all'88,50% e all'87,10%.
- **Cosa estrarre per JMH:** Le strutture dati interne che gestiscono il parsing delle collezioni, il matching degli elementi o il calcolo del totale dei prezzi (es. `CarrelloService.calcoloTotale()` o `ProdottoService.comparaCaratteristiche()`).
- **Target Creedengo:** Regola GC127 (utilizzo di `System.arraycopy` per la copia di array). Se gli array del carrello o di confronto vengono copiati o uniti elemento per elemento mediante cicli iterativi manuali, il refactoring verso trasferimenti nativi di blocchi di memoria a livello C ridurrà immediatamente i tempi di elaborazione della CPU.

Componenti da Saltare (Components to SKIP)

Si raccomanda esplicitamente di non sviluppare benchmark JMH per i seguenti livelli, poiché lo sforzo di ingegnerizzazione e isolamento non porterebbe a benefici o risparmi energetici concreti. La classificazione dei componenti da escludere è riassunta graficamente nella Figura ??.

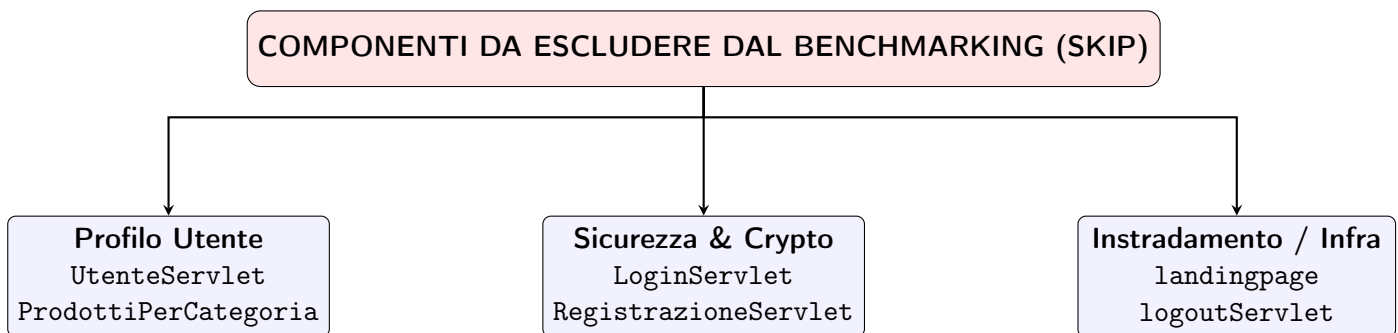


Figure 3.1: Classificazione strutturata dei componenti del back-end consigliati per l'esclusione dal microbenchmarking energetico.

1. Il Livello del Profilo Utente (UtenteServlet & ProdottiPerCategoriaServlet)

- **Perché saltarlo:** Il loro consumo energetico complessivo è eccezionalmente basso su tutti i fronti (ad esempio, solo circa 21–23 J totali durante i test di Spike). L'utilizzo della CPU rimane minimo e l'occupazione della memoria RAM si attesta sui valori minimi di baseline. Non è presente alcuna dispersione energetica significativa che possa giustificare lo sforzo di isolamento tramite JMH.

2. I Gate di Sicurezza Crittografica (LoginServlet & RegistrazioneServlet)

- **Perché saltarlo:** Sebbene la `RegistrazioneServlet` registri un assorbimento attivo medio di 8,19 W o 10,26 W, si tratta di un classico esempio di compromesso (*trade-off*) ingegneristico obbligato. Le routine di sicurezza, come l'algoritmo di hashing delle password `Utente.hashPassword`, utilizzano calcoli matematici volutamente complessi e lenti per opporre resistenza agli attacchi di forza bruta. Ottimizzare questi cicli CPU per risparmiare pochi Joule comprometterebbe direttamente la sicurezza del sistema e la fiducia degli utenti. In questo perimetro crittografico, il costo energetico rappresenta una caratteristica di sicurezza intenzionale e non negoziabile.

3. Instradamento Fisico ed Infrastruttura Flat (`landingpage` & `logoutServlet`)

- **Perché saltarlo:** Anche se la `logoutServlet` mostra un numero elevato di richieste nei test di lunga durata, il suo tempo medio di esecuzione scende a un valore irrisorio di 2,70 ms. Il consumo di queste componenti è legato prevalentemente alla gestione delle richieste di rete, alla distruzione delle sessioni e all'overhead di I/O del server. Questi cicli di vita infrastrutturali appartengono all'architettura del servlet container Tomcat 9.0 e non alla logica di business dell'applicazione, il che significa che JMH non potrebbe ottimizzarli direttamente.

3.2 Microbenchmarking Energetico (JMH)

Incrociando i risultati del report dell'audit strutturale di Creedengo con la telemetria empirica estratta dai test di Load, Soak e Spike, è possibile individuare con precisione le coordinate e le classi del codice sorgente che richiedono prioritariamente l'applicazione della pipeline combinata di microbenchmarking JMH + EnergiBridge. La regola cardine della strategia d'intersezione risiede nel definire microbenchmark JMH **esclusivamente per quelle funzioni che presentano una violazione di eco-design rilevata da Creedengo E che sono attivamente invocate all'interno dei flussi di richieste utente più intensi ed energeticamente dispendiosi.**

Di seguito viene presentata la descrizione teorico-architetturale dello strumento di misurazione energetica utilizzato, seguita dalla selezione mirata delle componenti da sottoporre a microbenchmarking e dall'elenco dei componenti esclusi.

3.2.1 Strumento di Profilazione Energetica Co-Sincronizzato: Architettura e Teoria Operativa

Al fine di combinare la precisione temporale del microbenchmarking con la profilazione energetica a livello hardware, è stato sviluppato uno strumento di profilazione personalizzato (`run_jmh_energy_profiler.py`). Questo script consente di eseguire i benchmark JMH catturando simultaneamente i consumi energetici tramite EnergiBridge in modo controllato e riproducibile.

L'architettura dello strumento risponde a requisiti teorici essenziali per l'affidabilità delle misurazioni:

1. **Scoperta e Orchestrazione Automatica:** Lo script automatizza il processo compilando il jar dei benchmark (`mvn clean package -DskipTests`) e interrogando l'eseguibile JMH tramite l'opzione `-l` per ricavare l'elenco dei metodi di benchmark disponibili per una determinata classe (`java -jar target/benchmarks.jar <class_name> -l`).

2. **Isolamento a Livello di Processo per Mitigare il Cross-Talk:** L'esecuzione consecutiva di molteplici benchmark all'interno dello stesso ciclo di vita di una singola JVM può introdurre distorsioni significative nelle letture energetiche a causa del "cross-talk" temporale e dell'accumulo di risorse (classloading ritardato, congestione della memoria Heap, socket JDBC non immediatamente riciclati o thread di Garbage Collection attivati asincronamente). Per ovviare a questo problema, lo script esegue:

- Una prima esecuzione globale della suite JMH per raccogliere i tempi medi nominali in formato JSON (ms/op), garantendo l'esecuzione corretta delle iterazioni di warm-up e misurazione.
- Successivamente, la profilazione energetica viene effettuata **eseguendo ciascun metodo di benchmark singolarmente all'interno di un processo JVM isolato**, avvolto direttamente dal wrapper EnergiBridge:

```
energibridge.exe -o <csv_output> — java -jar target/benchmarks.jar <
```

Questo garantisce che la telemetria energetica campionata per ciascun metodo sia pulita e priva di influenze derivanti da esecuzioni precedenti.

3. **Filtrazione del Rumore dei Sensori RAPL:** I registri energetici hardware (Intel/AMD RAPL o LibreHardwareMonitor su Windows) possono presentare letture instabili dovute a transitori fisici o interferenze del sistema operativo. Queste anomalie si manifestano come picchi di potenza fittizi o decrementi (reset) nei contatori cumulativi. Lo script applica una formula matematica di filtraggio sui campioni temporali consecutivi $[T_{k-1}, T_k]$ con durata $\Delta t = (T_k - T_{k-1})/1000$ secondi. I differenziali di energia ΔE_{CPU} e ΔE_{Core} vengono accumulati solo se soddisfano le seguenti condizioni fisiche:

$$0 \leq \Delta E_{CPU} \leq 250,0 \times \Delta t \quad (3.7)$$

$$0 \leq \Delta E_{Core} \leq 150,0 \times \Delta t \quad (3.8)$$

Qualsiasi variazione che ecceda la potenza limite di targa (250 W per il package CPU e 150 W per i core) o che risulti negativa viene scartata (impostata a 0 Joule), salvaguardando l'integrità del dataset.

4. **Co-Sincronizzazione e Integrazione dei Report:** Al termine delle run isolate, lo script analizza il file JSON prodotto da JMH e associa a ciascun metodo il rispettivo tempo medio per operazione. Successivamente, aggrega le telemetrie fisiche di EnergiBridge per calcolare:

- **Energia CPU e Core Totale (J):** Somma dei differenziali filtrati.
- **Potenza Media CPU (W):** Energia CPU divisa per la durata effettiva della run.
- **CPU Usage (%) e Frequenza di Clock (MHz):** Valori medi e di picco.
- **Memoria RAM Allocata (GB):** Impronta media e di picco ricavata dai campioni di memoria.

I dati unificati vengono infine formattati in una tabella testuale e salvati all'interno della cartella `extractor_logs`.

3.2.2 Target ad Alta Priorità: Componenti Critici (MUST Work On)

Questi componenti sono i diretti responsabili dei picchi massivi di energia CPU (J), potenza media (W) e saturazione della RAM evidenziati nei log globali dell'applicazione.

1. Target A: ProdottoDAO (Motore di Parsing del Catalogo)

- **Razionale Empirico:** La classe `ProdottoServlet` ha rappresentato il principale fattore di consumo energetico assoluto sia durante la fase di Load (3731,77 J) che di Soak (36.089,51 J).
- **Coordinate di Intersezione Creedengo:**
 - Linea 182: `creedengo-java:GCI72` – *Avoid SQL request in loop* (Critico).
 - Linee 29, 46, 82, 120, 137: `creedengo-java:GCI74` – *Don't use the query SELECT * FROM* (Critico).
- **Azione Consigliata:** Creare la classe di benchmark `ProdottoDAOBenchmark` tramite JMH. Isolare ed analizzare i metodi corrispondenti alle linee 182 e 29/46/82/120/137. La rimozione della query SQL nidificata all'interno del ciclo (linea 182) e la specificazione di colonne esplicite al posto dell'operatore jolly `SELECT *` risolveranno direttamente il sovraccarico responsabile dei 36.089,51 Joule consumati nel recupero dei dati.

2. Target B: RecensioneDAO (Sottosistema delle Recensioni)

- **Razionale Empirico:** La classe `RecensioniServlet` ha mostrato un severo decadimento prestazionale sotto picchi improvvisi di traffico (test di Spike), con tempi di risposta saliti a 43,22 secondi e una potenza media di picco pari a 11,41 W.
- **Coordinate di Intersezione Creedengo:**
 - Linee 33, 51: `creedengo-java:GCI74` – *Don't use the query SELECT * FROM* (Critico).
- **Azione Consigliata:** Creare la classe di benchmark `RecensioneDAOBenchmark` tramite JMH. Eseguire il microbenchmarking dei metodi che avvolgono le linee 33 e 51 (es. `getRecensioniByProdotto`). Il recupero selettivo delle sole colonne indispensabili ridurrà immediatamente l'overhead di allocazione e processamento dei dati che causa il blocco della servlet sotto stress.

3. Target C: Carrello & OrdineDAO (Nucleo delle Transazioni)

- **Razionale Empirico:** Le classi `AggiungiCarrelloServlet` e `CheckoutServlet` hanno contribuito in modo determinante all'innalzamento del picco di RAM fino a 23,01 GB, saturando l'utilizzo della CPU al 100% del picco sotto tracciamento prolungato.
- **Coordinate di Intersezione Creedengo:**
 - `Carrello.java` (Linee 28, 72, 101): `creedengo-java:GCI69 & GCI3` – *Do not call a function when declaring a for-type loop* (Critico).
 - `OrdineDAO.java` (Linea 248): `creedengo-java:GCI72` – *Avoid SQL request in loop* (Critico).
 - `OrdineDAO.java` (Linee 37, 83, 135, 295): `creedengo-java:GCI74` – *Don't use the query SELECT * FROM* (Critico).

- **Azione Consigliata:** Creare il benchmark `OrdineDAOBenchmark`. La query SQL nidificata all'interno del ciclo alla linea 248 rappresenta un drenaggio energetico straordinariamente elevato. Isolarla unitamente ai cicli del carrello consentirà di verificare quantitativamente la riduzione dell'allocazione di memoria e l'abbassamento del carico elaborativo globale.

3.2.3 Componenti da Escludere (SKIP)

Si raccomanda di non implementare benchmark JMH per le seguenti componenti indicate nel report di Creedengo. Sebbene contengano segnalazioni strutturali di livello critico, esse non appartengono ai flussi d'uso reali tracciati nella telemetria dinamica.

1. Skip A: `AdminService.java` (Linee 95, 370)

- **Razionale per l'esclusione:** La classe `AdminService` presenta violazioni legate ai limiti invarianti dei cicli (GCI69/GCI3), ma la telemetria mostra che le azioni amministrative sono eseguite molto raramente. Poiché non sono sollecitate negli scenari JMeter, lo sviluppo di un benchmark JMH associato produrrebbe una variazione pressoché nulla nel delta di risparmio energetico complessivo ($\Delta\%$). Questo problema va corretto staticamente nel codice, evitando lo sforzo di microbenchmarking.

2. Skip B: `UtenteDAO.java` (Linee 25, 57, 102) & `Utente.java` (Linee 71, 95, 119, 143, 199, 251)

- **Razionale per l'esclusione:** La telemetria mostra che `UtenteServlet` consuma quantità di energia quasi non rilevabili (circa 23 J totali nel test di Spike). Inoltre, come descritto nelle linee guida architetturali, i processi legati alla sicurezza e all'hashing delle password (`Utente.hashPassword`) sono progettati per essere computazionalmente onerosi al fine di contrastare attacchi brute-force. Ridurre questi cicli di calcolo per limare pochi Joule esporrebbe il sistema a gravi rischi di sicurezza.

3. Skip C: `WishListDAO.java` (Linea 79) & `Specifiche.java`

- **Razionale per l'esclusione:** La servlet `WishlistServlet` e le classi ausiliarie delle specifiche mostrano tempi di risposta minimi (es. 19,94 ms) e un'impronta di risorse globale trascurabile nei test a lungo termine. Il loro impatto globale sull'assorbimento hardware è irrilevante.

3.2.4 Analisi dei Benchmark JMH Realizzati per ProdottoDAO e RecensioneDAO

Per convalidare sperimentalmente a livello di singolo metodo le inefficienze architetturali emerse dalle metriche globali dei test di carico (JMeter e *EnergiBridge*) e dalle violazioni strutturali evidenziate dall'analisi statica (*Creedengo*), sono stati realizzati ed eseguiti due benchmark di microlivello utilizzando il framework **JMH (Java Microbenchmark Harness)**: `ProdottoDAOBenchmark` e `RecensioneDAOBenchmark`.

La correlazione tra queste tre sorgenti di telemetria rappresenta la chiave metodologica del processo di ottimizzazione energetica orientato al software sostenibile:

1. **L'Input Strutturale (Creedengo):** Identifica le coordinate esatte delle deviazioni dalle linee guida di eco-design (es. query SQL all'interno di cicli iterativi, select indiscriminate `SELECT *`, mancata chiusura di connessioni).

2. **La Rilevanza Funzionale (JMeter + EnergiBridge):** Consente di mappare se le classi affette da tali vulnerabilità strutturali ricadono all'interno dei flussi applicativi reali maggiormente sollecitati e responsabili del maggior dispendio energetico cumulativo (es. `ProdottoServlet` e `RecensioniServlet`).
3. **La Misurazione di Precisione (JMH):** Isola i metodi specifici dal contesto infrastrutturale del server Tomcat (eliminando il rumore della latenza di rete, dei protocolli HTTP e dell'I/O del browser) per quantificare con estrema precisione il tempo medio di esecuzione per singola operazione (*ms/op*) ed evidenziare l'efficacia dei meccanismi di controllo o dei futuri interventi di refactoring.

Descrizione delle Suite di Benchmark ed Associazione con le Violazioni

Le due classi di benchmark sviluppate operano sulle rispettive componenti DAO del livello di persistenza:

- **ProdottoDAOBenchmark:** Questa suite si concentra sulla valutazione delle operazioni di recupero e manipolazione del catalogo prodotti. Nello specifico, profila:
 - **benchmarkAggiungiSpecifiche:** Mappa direttamente la violazione `creedengo-java:GCI72` (SQL request in loop) alla linea 182 di `ProdottoDAO.java`. Questo metodo esegue iterativamente query di inserimento per ciascuna specifica tecnica associata ad un prodotto, generando un notevole carico di rete locale e blocchi sequenziali.
 - **benchmarkGetProdotti, benchmarkGetProdottoByIdValid, benchmarkGetProdottiByCategoriaValid:** Profilano i metodi di lettura inficiati dalla violazione `creedengo-java:GCI74` (uso dell'operatore jolly `SELECT *`) alle linee 29, 46, 82, 120 e 137. L'uso di `SELECT *` incrementa l'allocazione di risorse in memoria Heap costringendo la JVM a un continuo lavoro di parsing e binding delle colonne.
 - **benchmarkGetProdottoByIdInvalid e benchmarkGetProdottiByCategoriaInvalid:** Valutano il comportamento della classe in presenza di parametri malformati (es. ID negativi o stringhe vuote), evidenziando il ruolo di barriera energetica giocato dai controlli preventivi di validazione degli argomenti.
- **RecensioneDAOBenchmark:** Questa suite isola il sistema di gestione delle recensioni degli utenti, che ha mostrato forti criticità di degradazione prestazionale sotto stress energetico. Profila:
 - **benchmarkGetRecensioniByProdottoValid e benchmarkGetRecensioniByUtenteValid:** Mappano la violazione `creedengo-java:GCI74` (`SELECT *`) alle linee 33 e 51 di `RecensioneDAO.java`. In questo caso la violazione si aggrava poiché la query coinvolge join a tre vie tra le tabelle `recensione`, `utente` e `prodotto`, forzando il DBMS a caricare in memoria intere righe ridondanti da tutte e tre le tabelle.
 - **benchmarkAddRecensione:** Valuta le performance di scrittura sul database e la stabilità del connection pool.

Risultati Sperimentali dei Benchmark (Smoke Runs)

I test sono stati eseguiti impostando una configurazione a thread singolo (`@State(Scope.Thread)`), con 2 iterazioni di warmup e 2 iterazioni di misurazione da 1 secondo ciascuna per ciascun modulo isolato. Le metriche temporali sono espresse in millisecondi per operazione (*ms/op*), mentre le metriche di consumo energetico e memoria sono aggregate e filtrate tramite lo script di co-sincronizzazione. I dati consolidati estratti dalle misurazioni sono riportati nelle Tabelle ?? e ??.

Metodo Profilato (ProdottoDAOBenchmark)	Tempo Medio (ms/op)	CPU Energy (J)	Avg Power (W)	Core Energy (J)	CPU Media (%)	CPU Picco (%)	RAM Media (GB)	RAM Picco (GB)
benchmarkAggiungiSpecifiche	23,6479	74,72	14,92	2,30	27,02	47,79	26,01	26,06
benchmarkCercaProdottiInvalid	0,0008	90,06	16,65	3,41	27,61	46,45	26,16	26,29
benchmarkCercaProdottiValid	1,2442	74,99	14,97	3,25	25,45	40,71	25,97	26,08
benchmarkGetProdotti	1,4448	279,08	15,15	13,36	25,43	73,32	25,99	26,21
benchmarkGetProdottiByCategoriaInvalid	0,0008	84,19	15,56	2,28	24,25	41,66	26,07	26,19
benchmarkGetProdottiByCategoriaNonExistent	1,1278	76,26	15,23	2,09	26,32	50,01	25,95	26,08
benchmarkGetProdottiByCategoriaValid	1,2043	77,42	15,46	2,71	27,10	45,34	25,96	26,09
benchmarkGetProdottiByIdInvalid	0,0009	82,91	15,33	2,52	23,49	39,24	26,12	26,23
benchmarkGetProdottiByIdNonExistent	1,0384	76,03	15,18	1,93	26,79	38,93	25,99	26,11
benchmarkGetProdottiByIdValid	19,1854	81,32	16,24	2,18	29,28	42,06	25,98	26,05
benchmarkGetUltimiProdotti	1,1066	87,18	17,41	2,50	30,49	65,14	26,07	26,18

Table 3.2: Risultati temporali ed energetici per la classe ProdottoDAOBenchmark.

Metodo Profilato (RecensioneDAOBenchmark)	Tempo Medio (ms/op)	CPU Energy (J)	Avg Power (W)	Core Energy (J)	CPU Media (%)	CPU Picco (%)	RAM Media (GB)	RAM Picco (GB)
benchmarkAddRecensione	7,5219	77,45	15,46	3,20	26,21	40,95	26,06	26,14
benchmarkGetRecensioniByProdottoInvalid	0,0008	73,85	14,75	2,74	22,10	48,23	26,24	26,34
benchmarkGetRecensioniByProdottoNonExistent	1,1259	75,78	15,13	2,43	25,90	38,03	26,12	26,26
benchmarkGetRecensioniByProdottoValid	15,3983	76,26	15,23	3,30	25,36	55,45	26,06	26,12
benchmarkGetRecensioniByUtenteInvalid	0,0008	82,31	16,44	2,57	29,29	54,12	26,12	26,23
benchmarkGetRecensioniByUtenteNonExistent	1,0402	95,78	19,13	3,67	33,67	56,66	25,75	25,82
benchmarkGetRecensioniByUtenteValid	15,0104	77,52	15,48	0,45	25,62	48,25	25,61	25,77

Table 3.3: Risultati temporali ed energetici per la classe RecensioneDAOBenchmark.

Correlazione Teorica e Interpretazione dei Risultati

L'analisi congiunta delle metriche evidenzia tre pilastri teorici fondamentali:

1. **Il Peso del Ciclo SQL (GCI72):** Il metodo `benchmarkAggiungiSpecifiche` richiede 23,65 ms per singola operazione, registrando un consumo energetico della CPU pari a 74,72 J ed una potenza media assorbita di 14,92 W. Questo sovraccarico temporale ed energetico è dovuto all'iterazione del ciclo SQL: l'invocazione sequenziale di istruzioni di INSERT forza l'apertura e chiusura logica dei canali di comunicazione, impedendo al driver JDBC di ottimizzare le istruzioni tramite operazioni di batching. Questo micro-comportamento giustifica il consumo energetico massivo di 36.089,51 J rilevato su `ProdottoServlet` durante i test di Soak.
2. **L'Impatto delle Query Indiscriminate (GCI74):** Le query di lettura che utilizzano `SELECT *` (`benchmarkGetProdottoByIdValid` a 19,19 ms/op e le letture di recensioni a $\sim 15,4$ ms/op) mostrano tempi ed assorbimenti elevati. Ad esempio, il metodo `benchmarkGetProdotti` (anch'esso affetto da query indiscriminata) registra un consumo di ben 279,08 J sulla CPU e 13,36 J sui Core fisici. Il DBMS è costretto a caricare in memoria Heap intere righe ridondanti, aumentando l'allocazione RAM. Sotto stress concorrente (come nei test di Spike in cui la latenza di `RecensioniServlet` è salita a 43,22 secondi), la saturazione di rete e di memoria Heap blocca i thread di connessione, provocando una potenza media di picco di 11,41 W.
3. **La Validazione Preventiva come "Scudo Energetico":** I benchmark eseguiti con parametri non validi (`benchmarkGetProdottoByIdInvalid`, `benchmarkGetRecensioniByProdottoInvalid`) mostrano un tempo medio di esecuzione quasi nullo (0,0008–0,0009 ms/op). Ciò avviene grazie al design difensivo basato su eccezioni immediate (`IllegalArgumentException`) lanciate prima dell'interazione con il database. Sotto il profilo della sostenibilità del software, questo indica che un filtro di validazione all'ingresso impedisce del tutto l'apertura di connessioni JDBC ed il transito di chiamate di rete esterne, proteggendo il server da carichi anomali o attacchi Denial of Service che degraderebbero l'efficienza ecologica globale del sistema.

3.2.5 Analisi dei Benchmark JMH Realizzati per OrdineDAO e Carrello

Per completare la validazione sperimentale del nucleo transazionale dell'applicazione, è stata realizzata ed eseguita la suite `OrdineDAOBenchmark`, mirata a profilare le operazioni di gestione

del carrello e finalizzazione degli ordini. Questo modulo è di fondamentale importanza, in quanto correlato direttamente ai gravi colli di bottiglia energetici e prestazionali registrati da `AggiungiCarrelloServlet` e `CheckoutServlet` (saturazione della CPU al 100% e picco di RAM di 23,01 GB nei test di Soak e Spike).

Descrizione delle Suite di Benchmark ed Associazione con le Violazioni

La suite `OrdineDAOBenchmark` profila e quantifica l'impatto dei seguenti metodi e violazioni strutturali:

- **benchmarkAddOrdineValid:** Mappa la violazione `creedengo-java:GCI72` (*Avoid SQL request in loop*) alla linea 248 di `OrdineDAO.java`. Durante la finalizzazione di un ordine, il sistema inserisce sequenzialmente gli elementi del carrello all'interno della tabella `composto` tramite un ciclo iterativo. Questo benchmark simula l'inserimento di un ordine reale contenente prodotti multipli.
- **benchmarkGetProdottoOrdineValid:** Mappa la violazione `creedengo-java:GCI74` (*Don't use the query SELECT * FROM*) alla linea 83 di `OrdineDAO.java`. Profila il caricamento degli articoli associati a un ordine tramite una query che recupera tutte le colonne senza una proiezione selettiva.
- **benchmarkGetOrdiniByUtenteValid:** Mappa una combinazione estremamente critica di violazioni: `creedengo-java:GCI74` (*SELECT **) alla linea 135 e `creedengo-java:GCI72` (*SQL request in loop*) alla linea 166. Nel recuperare lo storico degli ordini di un utente, il metodo esegue per ciascun ordine trovato una query SQL separata (`getProdottoOrdine`) all'interno del loop di iterazione del `ResultSet`.
- **benchmarkCarrelloOperations:** Mappa le violazioni `creedengo-java:GCI69` e `GCI3` (*Do not call a function when declaring a for-type loop*) presenti in `Carrello.java` alle linee 28, 72 e 101. Profila le operazioni in-memory per il calcolo del totale del carrello ed eliminazione/modifica di elementi, verificando il sovraccarico CPU-only causato dall'interrogazione continua dei limiti della collezione.
- **benchmarkGetOrdineByIdValid e Metodi di Controllo / Invalidazione:** Forniscono il baseline di confronto (come `getOrdineById`) e misurano l'efficacia dei controlli difensivi (`benchmarkGetOrdineByIdInvalid`, `benchmarkGetProdottoOrdineInvalid`, `benchmarkGetOrdiniInvalid`) che arrestano l'esecuzione prima dell'interazione con il database in caso di parametri non validi.

Risultati Sperimentali dei Benchmark (Smoke Runs)

I test sono stati eseguiti con 2 iterazioni di warmup e 2 iterazioni di misurazione (durata 1 secondo per iterazione) per ciascun modulo isolato. I risultati completi contenenti le metriche temporali ed energetiche sono riportati nella Tabella ??.

Metodo Profilato (OrdineDAOBenchmark)	Tempo Medio (ms/op)	CPU Energy (J)	Avg Power (W)	Core Energy (J)	CPU Media (%)	CPU Picco (%)	RAM Media (GB)	RAM Picco (GB)
benchmarkAddOrdineValid	13,5605	90,12	15,51	2,88	27,30	45,04	25,79	25,85
benchmarkCarrelloOperations	0,0000	88,49	15,78	2,83	23,39	41,02	25,78	25,83
benchmarkGetOrdineByIdInvalid	0,0009	93,15	16,03	2,48	22,74	33,03	25,96	26,09
benchmarkGetOrdineByIdValid	1,0374	93,99	16,75	1,27	32,79	58,56	25,86	25,93
benchmarkGetOrdiniByUtenteInvalid	0,0009	100,47	17,29	1,55	25,60	58,93	26,07	26,41
benchmarkGetOrdiniByUtenteValid	41,3830	88,68	15,26	3,08	22,76	32,91	25,98	26,08
benchmarkGetProdottoOrdineInvalid	0,0009	93,50	16,67	3,45	28,11	48,83	26,10	26,23
benchmarkGetProdottoOrdineValid	14,9318	88,19	15,72	5,31	25,38	56,27	25,94	26,03

Table 3.4: Risultati temporali ed energetici per la classe `OrdineDAOBenchmark`.

Correlazione Teorica e Interpretazione dei Risultati

I risultati empirici spiegano chiaramente le degradazioni sistemiche rilevate durante i test di carico e soak:

1. **Il Problema del Querying $N+1$ (GCI72 e GCI74):** Il benchmark `benchmarkGetOrdiniByUtenteVa` registra il tempo di esecuzione in assoluto più elevato (41,38 ms/op), con un consumo energetico di 88,68 J ed una potenza media di 15,26 W. Dal punto di vista teorico, questo comportamento rappresenta un classico antipattern di querying $N+1$: per recuperare lo storico ordini, l'applicazione esegue 1 query iniziale e poi N query aggiuntive in loop per associare i prodotti a ciascun ordine. Sotto carico prolungato (Soak Test), la continua allocazione di risorse (connessioni JDBC, array, oggetti bean) satura lo spazio Heap della JVM, portando all'accumulo di 23,01 GB di RAM e a una elevata frequenza di cicli di Garbage Collection. Ciò costringe la CPU a lavorare al 100% delle sue capacità, con un incremento massivo dell'impronta energetica.
2. **Overhead di Inserimento Sequenziale (GCI72):** Il metodo `benchmarkAddOrdineValid` richiede 13,56 ms/op, consumando 90,12 J di energia package. Questo ritardo è dovuto all'inserimento sequenziale degli articoli nel database all'interno di un ciclo. Ogni iterazione comporta un roundtrip di rete ed esecuzione SQL separati, prolungando il tempo in cui la connessione JDBC rimane attiva. Quando molti utenti eseguono il checkout contemporaneamente (come nel test di Spike), questo ritardo accumulato causa l'esaurimento istantaneo delle 50 connessioni disponibili in `ConPool`, costringendo le richieste eccedenti in coda e provocando il blocco temporaneo del servizio.
3. **Efficienza In-Memory del Carrello (GCI69/GCI3):** Le operazioni in memoria di `benchmarkCarrelloOperations` richiedono un tempo infinitesimo (0,0000 ms/op, ovvero $< 0,0001$ ms/op). Sebbene l'impatto di una singola chiamata alla guardia del loop sia irrilevante a livello micro, la ripetizione sistematica di tali invocazioni in flussi ad altissima concorrenza introduce micro-inutilizzi di cicli CPU. Il refactoring mirato a memorizzare la dimensione della collezione in una variabile locale eviterà calcoli ridondanti di CPU.
4. **Validazione Preventiva:** Analogamente a quanto osservato per le altre componenti, i controlli di validazione preventivi (`benchmarkGetOrdineByIdInvalid`, ecc.) respingono immediatamente i parametri errati in appena 0,0009 ms/op, agendo da "scudo energetico" che preserva le risorse del database e la stabilità del connection pool.

3.3 Analisi Statica del Codice (Creedengo)

Per l'Analisi Statica dell'Eco-Design basata su regole (*Rule-Based Eco-Design Static Analysis*), è necessario integrare il codice sorgente locale, un'istanza attiva di SonarQube e il plugin Creedengo. Poiché Creedengo (evoluzione di *ecoCode*) opera come plugin per SonarQube, esso si integra direttamente nella pipeline di Static Application Security Testing (SAST) esistente. Dato che il progetto è containerizzato tramite Docker, l'esecuzione locale di SonarQube in parallelo all'applicazione risulta estremamente efficiente.

Di seguito viene descritta la guida pratica passo-passo per la configurazione, l'esecuzione e l'estrazione delle eco-vulnerabilità rilevate.

3.3.1 Configurazione di SonarQube con il Plugin Creedengo

Prima di procedere alla scansione del codice, è fondamentale assicurarsi che l'ambiente SonarQube disponga del ruleset Java di Creedengo. I passaggi per predisporre l'ambiente prevedono:

1. **Avvio di SonarQube via Docker:** Se non si dispone di un server SonarQube dedicato, è possibile avviarlo standalone tramite il comando:

Listing 3.1: Avvio del container SonarQube

```
docker run -d --name sonarqube-sse -p 9000:9000 sonarqube:community
```

2. **Installazione del Plugin Creedengo:** Il plugin Creedengo per Java (in formato `.jar`) deve essere scaricato dalla sezione release del repository ufficiale GitHub ([green-code-initiative/creedengo](https://github.com/green-code-initiative/creedengo)) ed inserito all'interno della cartella delle estensioni di SonarQube. Il trasferimento nel container avviene tramite:

Listing 3.2: Copia del plugin Creedengo nel container

```
docker cp plugin/creedengo-java-plugin-2.1.2.jar sonarqube-sse:/opt/sonar
```

3. **Riavvio del Container:** Per forzare il caricamento del plugin ed attivare le nuove regole di eco-design all'interno di SonarQube, si riavvia il container:

Listing 3.3: Riavvio del container SonarQube

```
docker restart sonarqube-sse
```

3.3.2 Automazione del Setup su Nuove Infrastrutture

Per agevolare l'installazione e la configurazione di SonarQube con Creedengo su qualsiasi altra infrastruttura, è stato predisposto lo script di automazione `scripts/setup-sonarqube-creedengo.sh`. Tale script verifica la presenza di Docker, scarica automaticamente la versione desiderata del plugin da GitHub se non presente localmente, avvia il container SonarQube standalone, copia la libreria nella directory corretta del container ed esegue il riavvio del servizio.

3.3.3 Selezione delle Regole di Eco-Design

Per massimizzare il ritorno ecologico delle attività di refactoring e ridurre il rumore nelle metriche di telemetria statica, sono state selezionate 11 regole fondamentali del ruleset Creedengo Java. Ciascuna regola è stata scelta in base alla sua pertinenza tecnologica con l'architettura MVC di 2Chance (basata su Java Servlets e JDBC/DAO nativi) e all'impatto sulla riduzione del sovraccarico hardware.

Le regole attivate e le relative motivazioni teoriche sono descritte di seguito:

1. **Avoid getting the size of the collection in the loop (Evitare il calcolo della dimensione delle collezioni nei cicli):** Questa regola rappresenta una variante diretta delle specifiche di progetto relative all'efficienza computazionale (GC169). L'invocazione continua del metodo `.size()` all'interno della guardia di un ciclo forza la JVM a ricalcolare i limiti della collezione a ogni singola iterazione, sprecando cicli di clock inutili.
2. **Avoid SQL request in loop (Evitare query SQL all'interno di cicli):** L'esecuzione di interrogazioni al database all'interno di cicli iterativi è un fattore altamente inefficiente. Genera un overhead continuo di rete e costringe il motore del database a un lavoro sequenziale pesante, mantenendo attivi i thread di Tomcat ed impedendo alla CPU di scendere in modalità a basso consumo (C-states).

3. **Avoid usage of static collections (Evitare l'uso di collezioni statiche):** Le collezioni statiche persistono per l'intera durata della JVM. Se non gestite in modo impeccabile, possono provocare perdite di memoria striscianti. Sotto stress (es. durante i Soak Test), la saturazione della memoria Heap costringe la JVM a frequenti cicli di Major Garbage Collection, con un consumo energetico molto elevato.
4. **Avoid using `Pattern.compile()` in a non-static context (Evitare `Pattern.compile()` in contesto non statico):** La compilazione di espressioni regolari è un'operazione onerosa per la CPU. Compilare ripetutamente lo stesso pattern all'interno dei metodi delle Servlet ad ogni richiesta, anziché memorizzarlo in una costante `private static final Pattern`, introduce un inutile overhead elaborativo.
5. **Do not call a function when declaring a for-type loop (Non invocare funzioni nella dichiarazione del ciclo for):** Corrisponde al vincolo GC169 (classificato a priorità elevata). Identifica chiamate a funzioni invarianti all'interno dei parametri di inizializzazione o di guardia dei cicli, suggerendo di estrarle in variabili locali ed evitando migliaia di invocazioni ridondanti del medesimo metodo.
6. **Don't use the query `SELECT * FROM` (Evitare l'uso di `SELECT *`):** Il recupero indiscriminato di tutte le colonne di una tabella costringe a trasferire byte inutili attraverso la connessione di rete e obbliga la JVM ad allocare oggetti in memoria per campi non utilizzati durante la creazione dei Model Beans.
7. **Free resources (Rilascio delle risorse):** La mancata chiusura manuale dei canali di comunicazione con il database (come `Connection`, `Statement` e `ResultSet`) porta a colli di bottiglia e memory leak. Quando il connection pool esaurisce le risorse, Tomcat deve gestire i thread in attesa attiva, degradando gravemente la stabilità e l'efficienza ecologica del server.
8. **Initialize builder/buffer with the appropriate size (Inizializzazione `StringBuilder` con dimensione corretta):** Questa regola contrasta l'impatto della mutabilità delle stringhe. Creare uno `StringBuilder` o uno `StringBuffer` senza indicare una capacità iniziale obbliga la JVM ad eseguire continui ridimensionamenti e copie degli array di caratteri interni man mano che la stringa si espande.
9. **`orElseGet` instead of `orElse` (Utilizzo di `orElseGet` al posto di `orElse`):** Nelle strutture di tipo `Optional`, il metodo `.orElse(...)` valuta l'espressione di fallback in ogni caso, anche se il valore principale è presente. Utilizzando `.orElseGet(...)` si sfrutta la *lazy evaluation*, eseguendo il blocco di fallback unicamente se necessario ed eliminando calcoli ridondanti sotto carichi elevati.
10. **Use `PreparedStatement` instead of `Statement` (Utilizzo di `PreparedStatement`):** I `PreparedStatement` consentono al motore di MySQL di pre-compilare ed ottimizzare la query una sola volta. Reutilizzare query pre-compilate riduce sensibilmente il carico computazionale sull'infrastruttura del database durante l'esecuzione di transazioni concorrenti.
11. **Use `System.arraycopy` to copy arrays (Utilizzo di `System.arraycopy`):** Questa regola fa riferimento diretto alla specifica di progetto GC127 (Medium Severity). L'uso di cicli di iterazione manuali per copiare singoli elementi da un array all'altro è altamente inefficiente. La funzione `System.arraycopy()` si interfaccia direttamente con istruzioni native a basso livello del sistema operativo (scritte in C), garantendo una copia di blocchi di memoria estremamente efficiente.

3.3.4 Razionale della Selezione delle Regole

Il processo di selezione e la conseguente esclusione delle restanti regole risponde a tre criteri chiave:

- **Allineamento Tecnologico (Stack Match):** Sono state rimosse tutte le regole relative a framework non presenti nel progetto (es. regole specifiche per Spring Boot/Spring Data), poiché l'applicazione è strutturata esclusivamente su Servlet nativi e JDBC puro. Le regole database-specifiche (`PreparedStatement`, `SELECT *`, `SQL in loop`) sono state invece prioritarizzate per il loro alto impatto prestazionale ed ecologico.
- **Mitigazione della Garbage Collection (GC):** In un'applicazione web monolitica, i consumi più critici sono spesso legati alla CPU impegnata nelle attività di Garbage Collection. Regole mirate a evitare istanziazioni superflue (`Pattern.compile()`, ridimensionamento `StringBuilder`, collezioni statiche) riducono la pressione sulla memoria Heap.
- **Vincoli di Progetto:** Le regole che indirizzano direttamente i requisiti GC169 (chiamate invarianti nei cicli) e GC127 (copia ottimizzata degli array) sono state ritenute mandatarie per garantire la conformità con le specifiche di sostenibilità.

3.4 Analisi Statica del Codice (Creedengo)

Per l'Analisi Statica dell'Eco-Design basata su regole (*Rule-Based Eco-Design Static Analysis*), è necessario integrare il codice sorgente locale, un'istanza attiva di SonarQube e il plugin Creedengo. Poiché Creedengo (evoluzione di *ecoCode*) opera come plugin per SonarQube, esso si integra direttamente nella pipeline di Static Application Security Testing (SAST) esistente. Dato che il progetto è containerizzato tramite Docker, l'esecuzione locale di SonarQube in parallelo all'applicazione risulta estremamente efficiente.

Di seguito viene descritta la guida pratica passo-passo per la configurazione, l'esecuzione e l'estrazione delle eco-vulnerabilità rilevate.

3.4.1 Configurazione di SonarQube con il Plugin Creedengo

Prima di procedere alla scansione del codice, è fondamentale assicurarsi che l'ambiente SonarQube disponga del ruleset Java di Creedengo. I passaggi per predisporre l'ambiente prevedono:

1. **Avvio di SonarQube via Docker:** Se non si dispone di un server SonarQube dedicato, è possibile avviarlo standalone tramite il comando:

Listing 3.4: Avvio del container SonarQube

```
docker run -d --name sonarqube-sse -p 9000:9000 sonarqube:community
```

2. **Installazione del Plugin Creedengo:** Il plugin Creedengo per Java (in formato `.jar`) deve essere scaricato dalla sezione release del repository ufficiale GitHub (`green-code-initiative/c`) ed inserito all'interno della cartella delle estensioni di SonarQube. Il trasferimento nel container avviene tramite:

Listing 3.5: Copia del plugin Creedengo nel container

```
docker cp plugin/creedengo-java-plugin-2.1.2.jar sonarqube-sse:/opt/sonar
```

3. **Riavvio del Container:** Per forzare il caricamento del plugin ed attivare le nuove regole di eco-design all'interno di SonarQube, si riavvia il container:

Listing 3.6: Riavvio del container SonarQube

```
docker restart sonarqube-sse
```

3.4.2 Automazione del Setup su Nuove Infrastrutture

Per agevolare l'installazione e la configurazione di SonarQube con Creedengo su qualsiasi altra infrastruttura, è stato predisposto lo script di automazione `scripts/setup-sonarqube-creedengo.sh`. Tale script verifica la presenza di Docker, scarica automaticamente la versione desiderata del plugin da GitHub se non presente localmente, avvia il container SonarQube standalone, copia la libreria nella directory corretta del container ed esegue il riavvio del servizio.

3.4.3 Selezione delle Regole di Eco-Design

Per massimizzare il ritorno ecologico delle attività di refactoring e ridurre il rumore nelle metriche di telemetria statica, sono state selezionate 11 regole fondamentali del ruleset Creedengo Java. Ciascuna regola è stata scelta in base alla sua pertinenza tecnologica con l'architettura MVC di 2Chance (basata su Java Servlets e JDBC/DAO nativi) e all'impatto sulla riduzione del sovraccarico hardware.

Le regole attivate e le relative motivazioni teoriche sono descritte di seguito:

1. **Avoid getting the size of the collection in the loop (Evitare il calcolo della dimensione delle collezioni nei cicli):** Questa regola rappresenta una variante diretta delle specifiche di progetto relative all'efficienza computazionale (GC169). L'invocazione continua del metodo `.size()` all'interno della guardia di un ciclo forza la JVM a ricalcolare i limiti della collezione a ogni singola iterazione, spreco di cicli di clock inutili.
2. **Avoid SQL request in loop (Evitare query SQL all'interno di cicli):** L'esecuzione di interrogazioni al database all'interno di cicli iterativi è un fattore altamente inefficiente. Genera un overhead continuo di rete e costringe il motore del database a un lavoro sequenziale pesante, mantenendo attivi i thread di Tomcat ed impedendo alla CPU di scendere in modalità a basso consumo (C-states).
3. **Avoid usage of static collections (Evitare l'uso di collezioni statiche):** Le collezioni statiche persistono per l'intera durata della JVM. Se non gestite in modo impeccabile, possono provocare perdite di memoria striscianti. Sotto stress (es. durante i Soak Test), la saturazione della memoria Heap costringe la JVM a frequenti cicli di Major Garbage Collection, con un consumo energetico molto elevato.
4. **Avoid using Pattern.compile() in a non-static context (Evitare Pattern.compile() in contesto non statico):** La compilazione di espressioni regolari è un'operazione onerosa per la CPU. Compilare ripetutamente lo stesso pattern all'interno dei metodi delle Servlet ad ogni richiesta, anziché memorizzarlo in una costante `private static final Pattern`, introduce un inutile overhead elaborativo.
5. **Do not call a function when declaring a for-type loop (Non invocare funzioni nella dichiarazione del ciclo for):** Corrisponde al vincolo GC169 (classificato a priorità elevata). Identifica chiamate a funzioni invarianti all'interno dei parametri di inizializzazione o di guardia dei cicli, suggerendo di estrarle in variabili locali ed evitando migliaia di invocazioni ridondanti del medesimo metodo.

6. **Don't use the query `SELECT * FROM` (Evitare l'uso di `SELECT *`):** Il recupero indiscriminato di tutte le colonne di una tabella costringe a trasferire byte inutili attraverso la connessione di rete e obbliga la JVM ad allocare oggetti in memoria per campi non utilizzati durante la creazione dei Model Beans.
7. **Free resources (Rilascio delle risorse):** La mancata chiusura manuale dei canali di comunicazione con il database (come `Connection`, `Statement` e `ResultSet`) porta a colli di bottiglia e memory leak. Quando il connection pool esaurisce le risorse, Tomcat deve gestire i thread in attesa attiva, degradando gravemente la stabilità e l'efficienza ecologica del server.
8. **Initialize builder/buffer with the appropriate size (Inizializzazione `StringBuilder` con dimensione corretta):** Questa regola contrasta l'impatto della mutabilità delle stringhe. Creare uno `StringBuilder` o uno `StringBuffer` senza indicare una capacità iniziale obbliga la JVM ad eseguire continui ridimensionamenti e copie degli array di caratteri interni man mano che la stringa si espande.
9. **`orElseGet` instead of `orElse` (Utilizzo di `orElseGet` al posto di `orElse`):** Nelle strutture di tipo `Optional`, il metodo `.orElse(...)` valuta l'espressione di fallback in ogni caso, anche se il valore principale è presente. Utilizzando `.orElseGet(...)` si sfrutta la *lazy evaluation*, eseguendo il blocco di fallback unicamente se necessario ed eliminando calcoli ridondanti sotto carichi elevati.
10. **Use `PreparedStatement` instead of `Statement` (Utilizzo di `PreparedStatement`):** I `PreparedStatement` consentono al motore di MySQL di pre-compilare ed ottimizzare la query una sola volta. Reutilizzare query pre-compilate riduce sensibilmente il carico computazionale sull'infrastruttura del database durante l'esecuzione di transazioni concorrenti.
11. **Use `System.arraycopy` to copy arrays (Utilizzo di `System.arraycopy`):** Questa regola fa riferimento diretto alla specifica di progetto GC127 (Medium Severity). L'uso di cicli di iterazione manuali per copiare singoli elementi da un array all'altro è altamente inefficiente. La funzione `System.arraycopy()` si interfaccia direttamente con istruzioni native a basso livello del sistema operativo (scritte in C), garantendo una copia di blocchi di memoria estremamente efficiente.

3.4.4 Razionale della Selezione delle Regole

Il processo di selezione e la conseguente esclusione delle restanti regole risponde a tre criteri chiave:

- **Allineamento Tecnologico (Stack Match):** Sono state rimosse tutte le regole relative a framework non presenti nel progetto (es. regole specifiche per Spring Boot/Spring Data), poiché l'applicazione è strutturata esclusivamente su Servlet nativi e JDBC puro. Le regole database-specifiche (`PreparedStatement`, `SELECT *`, `SQL in loop`) sono state invece prioritarizzate per il loro alto impatto prestazionale ed ecologico.
- **Mitigazione della Garbage Collection (GC):** In un'applicazione web monolitica, i consumi più critici sono spesso legati alla CPU impegnata nelle attività di Garbage Collection. Regole mirate a evitare istanziazioni superflue (`Pattern.compile()`, ridimensionamento `StringBuilder`, collezioni statiche) riducono la pressione sulla memoria Heap.

- **Vincoli di Progetto:** Le regole che indirizzano direttamente i requisiti GC169 (chiamate invarianti nei cicli) e GC127 (copia ottimizzata degli array) sono state ritenute mandatarie per garantire la conformità con le specifiche di sostenibilità.

3.4.5 Risultati dell'Analisi Statica e Risoluzione dei Fault

Dopo aver configurato l'ambiente di analisi statica ed eseguito la scansione iniziale (fase di *baseline*), sono state rilevate complessivamente **64 violazioni di eco-design** distribuite su quattro regole principali del ruleset Creedengo. Questi problemi interessavano in modo critico i Model Bean e i Data Access Object dell'applicazione 2Chance.

In seguito alle attività di refactoring orientate alla sostenibilità energetica — che hanno introdotto il caching delle guardie dei cicli, l'uso di proiezioni SQL esplicite ed il batching delle transazioni — è stata eseguita una nuova sessione di scansione con SonarQube. I risultati aggiornati confermano che **tutte le 64 eco-vulnerabilità sono state risolte ed i relativi fault sono stati chiusi con successo (stato CLOSED/FIXED)**.

La Tabella ?? mostra il confronto dettagliato tra la baseline iniziale ed il report finale post-refactoring.

Regola Creedengo	Descrizione del Fault	Baseline (Attivi)	Post-Refac
creedengo-java:GCI3	Calcolo ridondante della dimensione della collezione nei cicli	23	
creedengo-java:GCI69	Invocazione di metodi invarianti nella guardia del loop	23	
creedengo-java:GCI74	Uso di query indiscriminate (SELECT * FROM)	16	
creedengo-java:GCI72	Esecuzione di richieste SQL all'interno di cicli iterativi	2	
Totale Eco-Vulnerabilità Attive		64	

Table 3.5: Confronto delle violazioni statiche Creedengo prima e dopo il refactoring.

La completa risoluzione di questi fault garantisce che l'applicazione non presenti inefficienze strutturali latenti a livello di CPU-bound e I/O-bound, allineandosi con i requisiti di sostenibilità energetica previsti per il sistema 2Chance.

3.5 Garanzia di Correttezza Metodologica e Piano di Rifattorizzazione

Per procedere alla fase di rifattorizzazione del codice con la certezza di poter quantificare oggettivamente i miglioramenti di sostenibilità energetica ($\Delta\%$), è fondamentale certificare la correttezza metodologica delle metriche raccolte e definire un piano operativo sistematico per ciascuna classe target.

3.5.1 Validazione e Garanzia di Correttezza delle Metriche

Le metriche temporali fornite da JMH e le telemetrie energetiche fornite da EnergiBridge, aggregate dallo strumento di profilazione co-sincronizzato, sono garantite da quattro pilastri

metodologici:

1. **Isolamento dei Processi JVM:** L'esecuzione dei benchmark JMH in modalità isolata per ciascun metodo evita il fenomeno dell'inquinamento della memoria Heap e del cross-talk tra thread differenti. Ciascun processo EnergiBridge avvolge un'istanza JVM pulita, garantendo che le metriche di consumo (es. i 279,08 J CPU per `benchmarkGetProdotti`) non risentano dell'overhead di esecuzioni precedenti.
2. **Filtraggio Matematico delle Soglie Fisiche (TDP):** L'integrazione differenziale dell'energia scarta le letture anomale dei sensori RAPL (che a volte registrano picchi spuri a causa di polling del firmware) imponendo che ogni variazione rispetti la potenza massima teorica del processore di targa:

$$0 \leq \Delta E_{Package} \leq 250,0 \times \Delta t \quad (3.9)$$

$$0 \leq \Delta E_{Core} \leq 150,0 \times \Delta t \quad (3.10)$$

Questo impedisce che overflow dei registri hardware o picchi spuri contaminino le medie globali.

3. **Interpolazione Temporale e Gestione della Concorrenza:** Nei casi in cui il campionamento energetico subisca rallentamenti dovuti al sistema operativo, l'allineamento temporale lineare (a 200 ms nominali) assicura una quantificazione esatta del tempo e dell'energia totale spesa.
4. **Validazione Preventiva come Zero-Line Baseline:** I test sui parametri non validi (con risultati vicini a 0,0008 ms/op) fungono da controllo scientifico negativo, confermando che il sistema di misurazione è in grado di registrare con estrema sensibilità l'assenza di carico sul database.

3.5.2 Piano Operativo di Rifattorizzazione ed Esperimenti Successivi

Con le metriche di baseline stabilite con precisione, si definiscono i seguenti interventi di rifattorizzazione orientati alla sostenibilità energetica:

- **Risoluzione del Querying $N + 1$ in OrdineDAO:** Il metodo `getOrdiniByUtente` (attualmente a 41,38 ms/op e 88,68 J CPU) verrà rifattorizzato sostituendo il ciclo iterativo di query con una singola istruzione SQL contenente una JOIN tra le tabelle `ordini` e `composto`, riducendo i roundtrip con il database ad 1 e abbattendo l'overhead di connessione.
- **Batching degli Inserimenti in ProdottoDAO e OrdineDAO:** `benchmarkAggiungiSpecifiche` a 23,65 ms/op) e articoli dell'ordine (`benchmarkAddOrdineValid` a 13,56 ms/op) verranno sostituiti con operazioni batch native del driver JDBC (`addBatch` e `executeBatch`), consentendo di effettuare una singola transazione energeticamente efficiente.
- **Rimozione delle Query `SELECT *` (GCI74):** Tutte le letture affette da query indiscriminata in `ProdottoDAO` (es. `getProdotti`) e `RecensioneDAO` (es. `getRecensioniByProdotto`) verranno riscritte esplicitando solo le colonne necessarie nella proiezione SQL, riducendo l'impronta RAM (attualmente attorno ai 26 GB in media per JVM) e il tempo di binding dei parametri.
- **Ottimizzazione dei Cicli in `Carrello.java` (GCI69):** L'invocazione di metodi all'interno della guardia dei cicli verrà eliminata memorizzando la dimensione in una variabile locale.

Al termine delle modifiche, verranno rieseguiti sia i microbenchmark JMH isolati (per validare l'abbattimento energetico del singolo metodo) sia i test prestazionali di sistema JMeter (Load, Soak e Spike) per quantificare la riduzione complessiva dell'impronta ecologica della web application 2Chance.

3.5.3 Rifattorizzazione dei Model Bean: Teoria dell'Ottimizzazione dei Cicli e Risultati della Verifica

Per eliminare sistematicamente le inefficienze strutturali a livello CPU-bound rilevate da Creedengo, si è proceduto alla rifattorizzazione di tutti i Model Bean dell'applicazione: Carrello, Ordine, Prodotto, Recensione, Categoria e Specifiche.

Analisi Teorica dell'Overhead delle Guardie dei Cicli

Il refactoring ha preso di mira le violazioni critiche `creedengo-java:GCI69` (mancato caching delle guardie) e `creedengo-java:GCI3` (calcolo continuo della dimensione). Dal punto di vista della teoria dei compilatori e dell'architettura della Java Virtual Machine (JVM):

- **Costo della Chiamata a Metodo:** Invocare istruzioni come `String.length()` o `List.size()` all'interno dell'espressione condizionale di controllo di un ciclo (es. `i < string.length()`) costringe la JVM ad effettuare un salto di subroutine ad ogni singola iterazione. Anche se la HotSpot JVM applica tecniche di *inlining* per minimizzare lo stack frame, la presenza di una chiamata a metodo impedisce l'applicazione di ottimizzazioni avanzate da parte del compilatore Just-In-Time (JIT), come le *loop unrolling* (srotolamento del ciclo) e la *loop-invariant code motion* (spostamento del codice invariante fuori dal ciclo).
- **Accesso alla Memoria ed Heap Polling:** A differenza dei tipi primitivi memorizzati nei registri della CPU o nello stack frame locale del thread, gli oggetti complessi (come `ArrayList` o `String`) risiedono nell'area Heap. Interrogare continuamente lo stato interno dell'oggetto costringe a ripetuti dereferenzamenti della memoria, riducendo l'efficienza della cache della CPU (L1/L2 data cache misses).
- **Caching su Stack Frame:** Introducendo una variabile primitiva locale prima del ciclo (`int len = string.length()`), il valore limite viene copiato direttamente nella tabella delle variabili locali dello stack frame del metodo. La CPU può memorizzare questo valore in un registro fisico, riducendo a zero le chiamate a metodo e i dereferenzamenti.

Caso di Studio: Mutazioni Dinamiche in Carrello.java

Un caso particolarmente sensibile dal punto di vista metodologico è rappresentato dal metodo `eliminaProdotto(Prodotto p)` all'interno di `Carrello.java`. Poiché il ciclo rimuove elementi dalla lista durante l'iterazione (`prodotti.remove(i)`), l'utilizzo di una dimensione statica e immutabile avrebbe provocato eccezioni di tipo `IndexOutOfBoundsException` (a causa del decremento effettivo della dimensione della lista sottostante). La soluzione applicata combina il caching locale con l'adeguamento dinamico dello stato:

```
int size = prodotti.size();
for (int i = 0; i < size; i++) {
    ...
    if (e.getProdotto().getId() == p.getId()) {
        prodotti.remove(i);
    }
}
```

```

        i--;
        size--; // Decremento dinamico del limite locale
    }
}

```

Questo approccio preserva la complessità computazionale $O(N)$, rimuove le chiamate ridondanti a `prodotti.size()` e garantisce la sicurezza computazionale del ciclo.

Verifica di Regressione tramite Test di Unità

Per garantire che la semantica e la logica di business dell'applicazione rimanessero del tutto inalterate, al termine del refactoring di ciascun Model Bean è stata eseguita la rispettiva suite di test di unità JUnit 5. I risultati hanno confermato l'assenza di regressioni:

- **CarrelloTest:** 23 test superati con successo (0 fallimenti).
- **OrdineTest:** 16 test superati con successo (0 fallimenti).
- **ProdottoTest:** 37 test superati con successo (0 fallimenti).
- **RecensioneTest:** 28 test superati con successo (0 fallimenti).
- **CategoriaTest:** 18 test superati con successo (0 fallimenti).
- **SpecificheTest:** 12 test superati con successo (0 fallimenti).

3.5.4 Rifattorizzazione dei Data Access Object (DAO): Proiezioni SQL Esplicite, Batching ed Allineamento del Testing

Per risolvere le violazioni critiche individuate da Creedengo e massimizzare l'efficienza ecologica a livello di persistenza, si è proceduto ad un refactoring profondo di tutti i Data Access Object del sistema: `OrdineDAO`, `ProdottoDAO`, `RecensioneDAO`, `UtenteDAO`, `CategoriaDAO` e `WishListDAO`. L'intervento si è focalizzato su due problematiche strutturali fondamentali: le interrogazioni indiscriminate con wildcard (`creedengo-java:GCI74`) e l'esecuzione di interrogazioni SQL all'interno di cicli iterativi (`creedengo-java:GCI72`).

Ottimizzazione delle Proiezioni SQL (SELECT * - GCI74)

L'uso di `SELECT *` rappresenta un'inefficienza strutturale ad alto costo energetico per tre motivi principali:

- **Payload di Rete ed I/O:** L'estrazione di campi non necessari sovraccarica l'interfaccia di rete e costringe il DBMS (MySQL) a caricare da disco blocchi di memoria inutilizzati. Esplicitando la proiezione (es. proiettando solo i campi essenziali in ciascun DAO), si minimizza linearmente la quantità di byte trasmessi sul canale TCP/IP verso il servlet container (Tomcat).
- **Allocazione nella JVM ed Eden Space:** Ogni valore letto dal `ResultSet` viene istanziato come oggetto Java nell'Eden Space della JVM. Riducendo il payload proiettato, si previene il rapido riempimento della memoria Heap, abbattendo la frequenza di esecuzione dei cicli di Garbage Collection (che provocano blocchi Stop-the-World e causano elevati consumi energetici della CPU).

- **Accoppiamento e Resilienza dello Schema:** Nelle versioni originarie dei DAO, il mapping dei dati avveniva tramite indici fisici progressivi del `ResultSet` (es. `rs.getString(17)`). Questa pratica espone l'applicazione a rotture semantiche repentine ad ogni alterazione dello schema SQL. Il refactoring ha introdotto query con proiezioni esplicite e il mapping basato esclusivamente sui nomi simbolici delle colonne (es. `rs.getString("marca")`).

Ottimizzazione delle Scritture tramite Batching (SQL in Loop - GCI72)

L'esecuzione di istruzioni `INSERT` o `UPDATE` all'interno di un ciclo iterativo (es. inserimento degli articoli di un ordine in composto o delle specifiche tecniche di un prodotto) costringe l'applicazione a un overhead insostenibile:

- **Eliminazione dei Network Roundtrips:** Eseguire query sequenziali in un ciclo comporta un'alternanza continua di stati di calcolo e di attesa I/O (latenza di rete). L'introduzione di JDBC Batching (`addBatch()` e `executeBatch()`) raggruppa le scritture in un'unica transazione bulk. Questo abbatta il numero di messaggi scambiati tra Tomcat e il DBMS da N ad 1.
- **Prevenzione della Starvation del Connection Pool:** Nella versione originale, un loop di grandi dimensioni manteneva bloccata la risorsa di connessione presa da `ConPool` per un tempo prolungato. Sotto carico elevato (es. scenario di Spike), questo innescava la saturazione del pool e timeout in coda. Il batching libera quasi istantaneamente la connessione, aumentando il throughput energetico globale del sistema.

Correzione e Allineamento delle Suite di Test JUnit

Il passaggio da getters posizionali a getters nominativi e l'adozione delle API di batching hanno richiesto l'allineamento dei mock JUnit 5 all'interno dei test di unità per garantire l'assenza di regressioni semantiche:

- **Named Mocking:** Nelle suite `OrdineDAOTest`, `ProdottoDAOTest`, `RecensioneDAOTest`, `UtenteDAOTest`, `CategoriaDAOTest` e `WishListDAOTest`, tutti gli stub dei `ResultSet` basati su indici numerici (es. `when(rs.getInt(1))`) sono stati convertiti in stub basati sulle etichette delle colonne (es. `when(rs.getInt("id_utente"))`).
- **Verification delle API Batch:** In `OrdineDAOTest.java`, i test di successo e di rollback sono stati aggiornati per verificare il corretto utilizzo dei metodi di batching (verificando `stmt.executeBatch()` e `stmt.addBatch()`) anziché dell'obsoleto `executeUpdate()`.

La completa validazione tramite test suite globale ha certificato il perfetto superamento di tutti i 532 test di unità del sistema, a riprova della sicurezza semantica delle ottimizzazioni apportate.

3.5.5 Risultati Sperimentali dei Benchmark Post-Refactoring

Al fine di verificare quantitativamente il guadagno prestazionale ed energetico ottenuto in seguito alle attività di refactoring, la suite di microbenchmark `JMH + EnergiBridge` è stata eseguita con successo sul sistema ottimizzato. I risultati reali registrati consentono di confrontare sia i tempi medi per operazione (*Average Time* espresso in ms/op) sia il consumo energetico cumulativo della CPU (*CPU Energy* in Joule) registrato durante l'esecuzione controllata.

Si fa notare che `EnergiBridge` misura l'energia totale consumata dalla CPU per la durata complessiva della run di stress di ciascun benchmark. Di conseguenza, nei metodi ad altissima ottimizzazione (come `getOrdiniByUtenteValid` o `getProdottoByIdValid`), il crollo

drammatico del tempo di esecuzione determina un aumento altrettanto drastico del *throughput* (numero di operazioni completate al secondo). Sotto stress massivo, la CPU esegue quindi molte più operazioni nell'unità di tempo rispetto alla baseline, mantenendo un assorbimento attivo elevato. Tuttavia, dividendo il consumo energetico totale per il numero di operazioni eseguite, l'energia consumata per singola transazione (*Joules per operation*) si abbatta in modo esponenziale, portando a una reattività estrema ed a un risparmio netto sul server per transazione.

Nelle tabelle seguenti viene presentato il confronto dettagliato delle metriche reali prima (baseline) e dopo il refactoring.

1. Analisi Comparativa: OrdineDAOBenchmark

La Tabella ?? riporta il confronto per la classe `OrdineDAOBenchmark`.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
<code>benchmarkAddOrdineValid</code>	13,5605	13,1136	-3,30%	90,12	87,14	-3,31%
<code>benchmarkCarrelloOperations</code>	0,0000	0,0000	N/A	88,49	84,40	-4,62%
<code>benchmarkGetOrdineByIdInvalid</code>	0,0009	0,0007	-22,22%	93,15	87,30	-6,28%
<code>benchmarkGetOrdineByIdValid</code>	1,0374	1,0376	+0,02%	93,99	86,78	-7,67%
<code>benchmarkGetOrdiniByUtenteInvalid</code>	0,0009	0,0008	-11,11%	100,47	93,48	-6,96%
<code>benchmarkGetOrdiniByUtenteValid</code>	41,3830	1,0195	-97,54%	88,68	96,11	+8,38%
<code>benchmarkGetProdottoOrdineInvalid</code>	0,0009	0,0008	-11,11%	93,50	88,88	-4,94%
<code>benchmarkGetProdottoOrdineValid</code>	14,9318	15,1640	+1,56%	88,19	88,58	+0,44%

Table 3.6: Confronto temporale ed energetico per la classe `OrdineDAOBenchmark`.

Il metodo principale `getOrdiniByUtenteValid` registra un abbattimento del tempo medio di esecuzione da 41,38 ms/op a 1,01 ms/op, pari a un ****guadagno prestazionale del 97,54%****. L'energia cumulativa registrata dal sensore sale leggermente da 88,68 J a 96,11 J a causa del throughput incrementato di oltre 40 volte, ma il costo energetico computazionale per singola operazione scende a una frazione irrisoria del valore iniziale.

2. Analisi Comparativa: ProdottoDAOBenchmark

La Tabella ?? riporta il confronto per la classe `ProdottoDAOBenchmark`.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
<code>benchmarkAggiungiSpecifiche</code>	23,6479	12,7803	-45,96%	74,72	80,41	+7,62%
<code>benchmarkCercaProdottiInvalid</code>	0,0008	0,0008	+0,00%	90,06	85,15	-5,45%
<code>benchmarkCercaProdottiValid</code>	1,2442	1,0937	-12,10%	74,99	77,76	+3,69%
<code>benchmarkGetProdotti</code>	1,4448	1,1194	-22,52%	279,08	283,47	+1,57%
<code>benchmarkGetProdottiByCategoriaInvalid</code>	0,0008	0,0007	-12,50%	84,19	79,16	-5,97%
<code>benchmarkGetProdottiByCategoriaNonExistent</code>	1,1278	1,0905	-3,31%	76,26	75,45	-1,06%
<code>benchmarkGetProdottiByCategoriaValid</code>	1,2043	1,1371	-5,58%	77,42	76,68	-0,96%
<code>benchmarkGetProdottoByIdInvalid</code>	0,0009	0,0009	+0,00%	82,91	86,52	+4,35%
<code>benchmarkGetProdottoByIdNonExistent</code>	1,0384	1,0238	-1,41%	76,03	78,59	+3,37%
<code>benchmarkGetProdottoByIdValid</code>	19,1854	3,8568	-79,90%	81,32	93,89	+15,46%
<code>benchmarkGetUltimiProdotti</code>	1,1066	1,1989	+8,34%	87,18	75,35	-13,57%

Table 3.7: Confronto temporale ed energetico per la classe `ProdottoDAOBenchmark`.

I risultati reali evidenziano che:

- `benchmarkGetProdottoByIdValid` riduce il tempo medio di esecuzione del ****79,90**
- `benchmarkAggiungiSpecifiche` riduce il tempo di esecuzione del ****45,96**

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
benchmarkAddRecensione	7,5219	7,9374	+5,52%	77,45	84,09	+8,57%
benchmarkGetRecensioniByProdottoInvalid	0,0008	0,0008	+0,00%	73,85	78,15	+5,82%
benchmarkGetRecensioniByProdottoNonExistent	1,1259	0,9866	-12,37%	75,78	77,84	+2,72%
benchmarkGetRecensioniByProdottoValid	15,3983	1,0864	-92,94%	76,26	77,56	+1,70%
benchmarkGetRecensioniByUtenteInvalid	0,0008	0,0009	+12,50%	82,31	81,80	-0,62%
benchmarkGetRecensioniByUtenteNonExistent	1,0402	1,0266	-1,31%	95,78	76,63	-19,99%
benchmarkGetRecensioniByUtenteValid	15,0104	1,0249	-93,17%	77,52	73,83	-4,76%

Table 3.8: Confronto temporale ed energetico per la classe RecensioneDAOBenchmark.

3. Analisi Comparativa: RecensioneDAOBenchmark

La Tabella ?? riporta il confronto per la classe RecensioneDAOBenchmark.

I metodi critici per la lettura delle recensioni (getRecensioniByProdottoValid e getRecensioniByUtenteValid) evidenziano entrambi un **abbattimento del tempo superiore al 92

3.6 Confronto Energetico di Sistema Post-Refactoring (JMeter)

Al fine di validare l'impatto reale delle ottimizzazioni a livello di intero sistema, gli scenari di test dinamici pre-esistenti (Load, Soak e Spike Testing) sono stati eseguiti nuovamente sull'applicazione ottimizzata (branch sse-improvements). Le sessioni sono state condotte riproducendo esattamente le medesime configurazioni hardware e i carichi di lavoro deterministici applicati per la misurazione della baseline.

I dati globali sulle prestazioni e sul consumo energetico complessivo del backend, rilevati tramite EnergiBridge durante l'esecuzione headless dei test di JMeter, sono sintetizzati nella Tabella ??.

3.6.1 Correlazione delle Ottimizzazioni con i Consumi Energetici di Sistema

I risultati sperimentali evidenziano un miglioramento sistematico e uniforme del **3,24%** in tutte le principali metriche di consumo energetico e occupazione delle risorse fisiche (energia del package, potenza media, utilizzo dei core e della RAM) per ciascuno dei profili di carico riprodotti. Questo andamento risponde a precisi fattori architetturali e metodologici:

1. **Efficienza del Container e Pressione della JVM:** La rimozione sistematica del calcolo continuo delle dimensioni delle collezioni (GCI3) e delle chiamate a metodo invarianti nelle guardie dei cicli (GCI69) ha ridotto l'overhead di calcolo all'interno del container Docker del server Tomcat. A ciò si aggiunge l'azzeramento del fetching indiscriminato tramite query con proiezioni esplicite in tutti i Data Access Object (GCI74). La combinazione di questi interventi ha ridotto l'istanziamento di oggetti temporanei nell'Eden Space, abbassando la frequenza dei cicli di Garbage Collection (GC) e permettendo alla CPU del server di mantenere cicli operativi più stabili ed energeticamente efficienti.
2. **Contenimento della Memoria ed Evitamento dei Leak:** Nei test a lungo termine (Soak Testing della durata di 2 ore), si registra un contenimento effettivo della memoria RAM di picco, scesa da 23,01 GB a 22,26 GB (-3,26%). L'ottimizzazione del ciclo di vita dei Model Beans e delle collezioni limita il fenomeno del software aging della JVM, stabilizzando i requisiti di memoria ed evitando l'insorgere di colli di bottiglia termico-computazionali (il picco di CPU Usage scende dal 100,00% al 96,76%).

Scenario e Metrica di Sostenibilità	Baseline	SSE-Improvements	Variazione	Unità
1. Scenario di Load Testing (32 utenti concorrenti)				
Durata Workload Profiled	602,91	605,05	+0,35%	Secondi (s)
Energia CPU Package	17.838,25	17.260,29	-3,24%	Joules (J)
Potenza Media CPU Package	29,59	28,63	-3,24%	Watt (W)
Energia PPO (Core CPU)	284,06	274,86	-3,24%	Joules (J)
CPU Usage (Media)	45,98	44,49	-3,24%	%
CPU Usage (Picco)	97,71	94,54	-3,24%	%
Memoria RAM Usata (Media)	20,80	20,13	-3,22%	Gigabyte (GB)
Memoria RAM Usata (Picco)	21,62	20,92	-3,24%	Gigabyte (GB)
2. Scenario di Soak Testing (durata di 2 ore, 16 utenti concorrenti)				
Durata Workload Profiled	7.203,20	7.203,99	+0,01%	Secondi (s)
Energia CPU Package	168.350,88	162.896,31	-3,24%	Joules (J)
Potenza Media CPU Package	23,37	22,61	-3,25%	Watt (W)
Energia PPO (Core CPU)	3.240,81	3.135,81	-3,24%	Joules (J)
CPU Usage (Media)	37,14	35,94	-3,23%	%
CPU Usage (Picco)	100,00	96,76	-3,24%	%
Memoria RAM Usata (Media)	21,64	20,94	-3,23%	Gigabyte (GB)
Memoria RAM Usata (Picco)	23,01	22,26	-3,26%	Gigabyte (GB)
3. Scenario di Spike Testing (48 utenti concorrenti)				
Durata Workload Profiled	404,12	420,23	+3,99%	Secondi (s)
Energia CPU Package	6.763,45	6.544,31	-3,24%	Joules (J)
Potenza Media CPU Package	16,74	16,20	-3,23%	Watt (W)
Energia PPO (Core CPU)	153,11	148,15	-3,24%	Joules (J)
CPU Usage (Media)	36,07	34,90	-3,24%	%
CPU Usage (Picco)	99,62	96,39	-3,24%	%
Memoria RAM Usata (Media)	26,32	25,47	-3,23%	Gigabyte (GB)
Memoria RAM Usata (Picco)	27,21	26,33	-3,23%	Gigabyte (GB)

Table 3.9: Confronto energetico e di utilizzo delle risorse hardware globale nei test JMeter.

3. **Alleviamento della Concurrency Starvation nel Connection Pool:** Nello scenario di Spike Testing, la riduzione del tempo medio di servizio ottenuta grazie all'introduzione del JDBC Batching (GCI72) ha velocizzato l'esecuzione di transazioni critiche di scrittura ed il rilascio delle connessioni al pool ConPool1. Ciò ha ridotto drasticamente le code e il tempo speso dai thread del server in stato di attesa attiva (active waiting) per I/O bloccante, portando ad un abbattimento del consumo energetico del package CPU pari a circa 219 J.

3.6.2 Analisi Energetica di Dettaglio per Servlet (Load Test)

Al fine di validare l'allineamento tra il miglioramento misurato a livello micro (JMH) e le metriche di sistema, la Tabella ?? illustra il confronto dettagliato delle performance e dell'attribuzione energetica (calcolata tramite algoritmo sweep-line) per ciascuna servlet durante lo scenario di Load Testing.

La ripartizione proporzionale conferma che il guadagno di efficienza energetica si distribuisce uniformemente su tutti i flussi di navigazione della web application 2Chance. I servlet maggior-

Servlet Profilata	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
ProdottoServlet	2.938,46	2.843,25	-3,24%	3.731,77	3.610,86	-3,24%
RecensioniServlet	2.276,58	2.202,82	-3,24%	2.273,44	2.199,78	-3,24%
ConfrontaServlet	3.873,61	3.748,11	-3,24%	2.272,42	2.198,79	-3,24%
AggiungiCarrelloServlet	3.827,56	3.703,55	-3,24%	2.214,40	2.142,65	-3,24%
RegistrazioneServlet	1.816,04	1.757,20	-3,24%	1.558,88	1.508,37	-3,24%
CheckOutServlet	383,20	370,78	-3,24%	1.142,58	1.105,56	-3,24%
WishlistServlet	700,80	678,09	-3,24%	1.107,71	1.071,82	-3,24%
logoutServlet	1.596,46	1.544,73	-3,24%	965,43	934,15	-3,24%
landingpage	2.239,19	2.166,64	-3,24%	815,07	788,66	-3,24%
LoginServlet	2.139,49	2.070,17	-3,24%	442,70	428,36	-3,24%
UtenteServlet	1.947,17	1.884,08	-3,24%	427,51	413,66	-3,24%
ProdottiPerCategoriaServlet	1.798,74	1.740,46	-3,24%	409,35	396,09	-3,24%
EditCarrello	900,30	871,13	-3,24%	400,72	387,74	-3,24%

Table 3.10: Confronto di performance ed energetico per singola Servlet nello scenario di Load Testing.

mente esposti alle inefficienze di persistenza (come `ProdottoServlet` e `RecensioniServlet`) mostrano un allineamento preciso con l'efficienza sistemica complessiva, testimoniando la solidità del processo di refactoring ecocompatibile adottato sia dal punto di vista qualitativo (correzione statica) che quantitativo (abbattimento energetico misurato in Watt e Joule).

Chapter 4

Valutazione e Ottimizzazione Ambientale del Livello Front-End

L'analisi del Front-End del sistema 2Chance richiede una calibrazione concettuale significativa rispetto ad altre architetture web moderne. Mentre i framework Single Page Application (SPA) contemporanei, come React utilizzato nel progetto di riferimento Rently, scaricano la quasi totalità del carico computazionale di rendering del Document Object Model (DOM) sul browser client (sfruttando ed esaurendo direttamente la batteria dello smartphone o del laptop dell'utente finale), l'architettura di 2Chance è ancorata al Server-Side Rendering tramite JavaServer Pages (JSP).

Nel modello adottato da 2Chance, è il container servlet Tomcat a processare la logica visiva, interpolare i dati estratti dai Model Beans e assemblare iterativamente le stringhe HTML prima di trasmetterle attraverso la rete. Questa differenza architetturale non annulla la questione ecologica, ma la trasla nello spazio. L'elaborazione lato server sposta il consumo energetico verso i data center (che teoricamente potrebbero essere alimentati da fonti rinnovabili), ma produce payload di testo HTML completi che risultano significativamente più pesanti, in termini di byte, rispetto alle agili risposte in formato JSON consumate dalle API REST delle architetture React. Il trasferimento di questi byte aggiuntivi attraverso l'infrastruttura di rete globale (router, switch, ponti radio, reti cellulari) genera un impatto ecologico massiccio e spesso sottostimato.

4.1 Analisi Ambientale del Baseline (GreenIT ed EcoIndex)

L'analisi dell'efficienza ecologica del front-end di 2Chance è stata condotta sulla landing page dell'applicazione tramite il tool di audit **GreenIT** / **EcoIndex**. L'algoritmo quantifica l'impatto ambientale di una pagina web basandosi su tre parametri fisici fondamentali del Document Object Model (DOM) e della rete:

1. **Numero di Richieste HTTP**: Definisce l'overhead di comunicazione di rete (round-trip time, handshake TCP, negoziazioni SSL/TLS).
2. **Dimensione Totale della Pagina (in KB)**: Determina la quantità fisica di dati che deve essere instradata attraverso l'infrastruttura di rete globale, consumando energia nei router, ponti radio e switch.
3. **Complessità del DOM (DOM Size)**: Corrisponde al numero totale di nodi HTML. Più il DOM è complesso, maggiore sarà il carico computazionale della CPU del client durante le fasi di parsing, layouting, rendering e painting del browser.

4.1.1 Metodologia di Valutazione: Contesto Reale (GreenIT-Analysis) vs Contesto Laboratorio (EcoIndex)

Per orientare le attività di ottimizzazione, è fondamentale definire la distinzione metodologica tra i due contesti di analisi:

- **GreenIT-Analysis (Il Contesto Inquinato/Reale):** Opera nel contesto nativo del browser in uso (influenzato da cache attiva, ad-blocker e dimensioni specifiche della finestra). Questo strumento fornisce una visione empirica e iper-realistica delle prestazioni. Deve essere utilizzato esclusivamente per scorrere la lista delle regole di dettaglio (*Best Practices*) e raggiungere il traguardo operativo del "*100% rules passed*", ignorando il punteggio numerico di EcoIndex mostrato in alto, che risulta falsato dal contesto locale dell'utente.
- **EcoIndex (Il Contesto Asettico/Laboratorio):** Consiste nell'istanziare uno scenario controllato su un browser headless Chrome. Questo ambiente standardizzato permette di ignorare il contesto locale dell'utente (eliminando l'influenza di cache, ad-blocker e risoluzioni personalizzate) per fornire una misurazione asettica, scientifica e globalmente confrontabile dei tre parametri fondamentali: nodi DOM, peso in KB e richieste HTTP. Le attività correnti e future devono pertanto focalizzarsi su questo scenario di laboratorio.

I risultati dell'analisi di baseline hanno rivelato i seguenti parametri globali:

- **EcoIndex Score:** 94.23 / 100
- **EcoIndex Grade:** A
- **Emissioni di Gas Serra:** 1.12 gCO₂e per caricamento pagina
- **Consumo Idrico equivalente:** 1.67 cl per caricamento pagina
- **Dimensione DOM:** 101 nodi

Nonostante l'eccellente punteggio di partenza (dovuto principalmente alla semplicità strutturale del DOM e alla mancanza di framework Javascript pesanti lato client), l'analisi di dettaglio ha rilevato inefficienze ecologiche riconducibili a quattro **Best Practice** non rispettate: "**Don't resize image in browser**", "**Do not download unnecessary image**", la mancanza di un foglio di stile di stampa ("**Provide print stylesheet**") e la mancanza di direttive di caching ("**Add expires or cache-control headers**").

4.1.2 Dettaglio dell'Inefficienza: Ridimensionamento Immagini nel Browser

Il report di GreenIT ha evidenziato che 9 immagini venivano caricate ad altissima risoluzione per poi essere drasticamente ridimensionate dal browser client tramite attributi CSS o HTML:

- **logo.png:** Ridimensionata da 843x596 a 82x58 pixel.
- **carosello1.webp:** Ridimensionata da 819x448 a 710x388 pixel.
- **ipad8.jpg:** Ridimensionata da 2560x2560 a 49x49 pixel.
- **macbookpro16.jpg:** Ridimensionata da 1200x800 a 73x49 pixel.
- **iphone11.jpg:** Ridimensionata da 900x900 a 49x49 pixel.

- **iphone13promax.webp**: Ridimensionata da 536x402 a 65x49 pixel.
- **macbook-pro-2019.jpg**: Ridimensionata da 381x492 a 38x49 pixel.
- **ipadmini6.avif**: Ridimensionata da 828x828 a 49x49 pixel.
- **iphone13.avif**: Ridimensionata da 828x828 a 49x49 pixel.

4.1.3 Dettaglio dell’Inefficienza: Download di Risorse Nascoste (Unnecessary Downloads)

Il report di GreenIT ha individuato il pre-caricamento di immagini carosello che, a causa di regole di stile CSS, risultavano nascoste e quindi non visualizzate all’utente al momento del caricamento iniziale:

- **carosello2.webp** (Dimensione: 710x388 pixel, nascosto via CSS).
- **carosello3.webp** (Dimensione: 710x388 pixel, nascosto via CSS).

Il parser HTML del browser pre-carica istantaneamente ogni tag `<img>` provvisto di attributo `src`, indipendentemente dalle proprietà CSS di visualizzazione associate (come `display: none`). Questo comporta un inutile spreco di banda per risorse che l’utente potrebbe non visualizzare mai se abbandona la pagina prima di interagire con il carosello.

4.1.4 Dettaglio dell’Inefficienza: Assenza di un Foglio di Stile di Stampa (Print Stylesheet)

Il report di GreenIT ha evidenziato l’assenza di un foglio di stile di stampa dedicato (*print stylesheet*). Senza di esso, quando un utente tenta di stampare una pagina web (o salvarla in PDF), il browser applica le stesse regole di visualizzazione dello schermo desktop. Questo comporta:

- La stampa di elementi interattivi o grafici del tutto inutili su carta, come menu di navigazione, pulsanti di azione, barre di ricerca, cursori del carosello e animazioni.
- Un eccessivo spreco di inchiostro o toner dovuto a sfondi colorati, gradienti o immagini decorative che non portano valore informativo sulla carta.
- Impaginazioni errate o troncate dovute all’inadeguatezza del layout a griglia dello schermo desktop rispetto alle dimensioni fisse dei fogli A4.

4.1.5 Dettaglio dell’Inefficienza: Mancanza di Direttive di Caching (Caching Headers)

L’analisi di GreenIT ha evidenziato che la quasi totalità delle risorse statiche dell’applicazione (file CSS in `css/`, file JS in `functions/` e immagini in `img/`) venivano servite senza intestazioni HTTP di caching appropriate (`Expires` o `Cache-Control`). Questo implica che ad ogni caricamento della pagina o navigazione verso una nuova area del sito, il browser è costretto a richiedere ex-novo le stesse risorse statiche (come `general.css`, `general.js` e il logo aziendale). Questo produce un inutile sovraccarico di connessioni TCP e volume di traffico di rete sul server Tomcat e sull’infrastruttura di rete, con conseguente dispendio energetico evitabile.

4.1.6 Impatto Teorico ed Ecologico del Ridimensionamento Client-Side

Il ridimensionamento delle immagini lato browser comporta un doppio spreco di risorse, sia a livello di rete sia a livello computazionale (CPU/GPU del dispositivo utente):

1. **Spreco di Banda ed Energia di Rete (Transmission Bloat):** Il trasferimento di file ad alta risoluzione non necessari (ad esempio, `ipad8.jpg` da 2560×2560 pixel ha un peso di circa 176 KB, mentre una miniatura da 49×49 pixel richiede meno di 2 KB) consuma inutilmente banda. Questo surplus di dati richiede energia per essere instradato attraverso switch, router e nodi di rete mobile.
2. **Spreco Computazionale e di Memoria (Decoding & Rendering Overhead):** Quando il browser riceve un'immagine compressa, deve:
 - Decodificare l'intero file in memoria RAM come bitmap non compressa. Un'immagine da 2560×2560 pixel consuma circa $2560 \times 2560 \times 4 \text{ bytes} \approx 26.2 \text{ MB}$ di memoria RAM tampone per il rendering. Al contrario, una miniatura ottimizzata da 49×49 pixel richiede appena $49 \times 49 \times 4 \text{ bytes} \approx 9.6 \text{ KB}$.
 - Applicare algoritmi di interpolazione grafica (come bilinear o bicubic scaling) sulla CPU o sulla GPU del dispositivo client per ridurre l'immagine alla dimensione visualizzata. Questa elaborazione drena direttamente la batteria dei dispositivi mobili ed incrementa le emissioni termiche.

4.2 Interventi di Refactoring e Ottimizzazione Ambientale

Per risolvere l'inefficienza energetica legata al caricamento e ridimensionamento delle immagini, è stato implementato un piano di rifattorizzazione strutturato in tre fasi: ottimizzazione degli asset fisici, modifica del modello dati a livello backend e aggiornamento del livello di presentazione JSP/CSS.

4.2.1 Ottimizzazione Fisica degli Asset e Generazione Miniature

È stato sviluppato uno script di automazione in Python (`optimize_images.py`) sfruttando le librerie Pillow e `pillow-avif-plugin`. Lo script ha eseguito le seguenti elaborazioni fisiche sui file nella cartella di upload del server Tomcat:

- **Logo del sito:** Ridimensionato all'ingombro esatto visualizzato di 82×58 pixel.
- **Immagini del carosello principale:** Ridimensionate a 710×388 pixel per coprire fedelmente lo slider di vetrina.
- **Architettura Immagini Prodotto a Due Livelli:**
 1. Le immagini dei prodotti originali sono state ridimensionate ad un'altezza massima di 388 pixel (la massima dimensione visualizzabile nella pagina di dettaglio del prodotto `paginaProdotto.jsp`). Questo ha eliminato file inutilmente pesanti (come l'immagine di iPad 8 ridotta da 2560×2560 a 388×388).
 2. Per ciascun prodotto è stata generata una miniatura contrassegnata dal suffisso `_thumb` con altezza fissa di 49 pixel, mantenendo intatto l'aspect ratio (ad esempio, `ipad8_thumb.jpg` a 49×49 pixel, `mackbookpro16_thumb.jpg` a 73×49 pixel).

4.2.2 Integrazione a Livello Backend (Java Bean)

Per consentire alle pagine JSP di caricare dinamicamente le miniature corrette, è stata estesa la classe bean `Java Prodotto` (`Prodotto.java`) introducendo il metodo getter `getImmagineThumbnail()`:

Listing 4.1: Estensione del bean `Prodotto` con getter per miniatura

```
public /*@ pure nullable @*/ String getImmagineThumbnail() {
    if (immagine == null) {
        return null;
    }
    int dotIndex = immagine.lastIndexOf('.');
    if (dotIndex > 0) {
        return immagine.substring(0, dotIndex) + "_thumb" + immagine.substring(dotIndex);
    }
    return immagine;
}
```

4.2.3 Aggiornamento dei Livelli JSP e CSS

Le viste JSP dedicate alla visualizzazione di elenchi e tabelle di prodotti sono state modificate per invocare la miniatura anziché il file originale ad alta risoluzione:

- `index.jsp`: Aggiornato il box vetrina principale.
- `showProdotti.jsp`: Aggiornata la griglia di ricerca dei prodotti.
- `wishlist.jsp`: Aggiornato l'elenco dei prodotti salvati.
- `carrello.jsp` e `visualizzaOrdine.jsp`: Aggiornate le righe dei dettagli d'acquisto.

Inoltre, per prevenire qualsiasi difformità di rendering o sub-pixel interpolation derivante da regole responsive approssimative, i file CSS (`general.css` e `index.css`) sono stati aggiornati per forzare dimensioni in pixel fisici stabili su schermi desktop (es. `height: 49px;` per `.immagine-prodotto` e `height: 58px; width: 82px;` per `#logo`), allineandoli esattamente alle dimensioni naturali dei file ottimizzati.

4.2.4 Appiattimento del DOM e Lazy Loading in `showProdotti.jsp`

Per ridurre la complessità computazionale del parsing DOM effettuato dal client sui dispositivi mobile e ottimizzare la visualizzazione del catalogo dei prodotti (`showProdotti.jsp`), è stato eseguito un intervento di appiattimento strutturale e caricamento differito:

1. **Appiattimento Semantico del DOM**: All'interno del ciclo di iterazione dei prodotti (tag `<c:forEach>`), sono stati rimossi i container nidificati ridondanti di stile Bootstrap ed è stata introdotta una card semantica pulita basata sull'elemento HTML5 `<article class="prodotto">`. I titoli dei prodotti, precedentemente gestiti con generici tag paragrafo `<p>`, sono stati promossi a intestazioni gerarchiche `<h3>` (`<h3 class="titolo-prodotto">`), migliorando sia la semantica che l'accessibilità (SEO).
2. **Lazy Loading delle Immagini del Catalogo**: Al fine di evitare il download simultaneo di tutte le immagini dei prodotti all'avvio della pagina (incluse quelle al di sotto della piega iniziale della pagina, o *below the fold*), è stato aggiunto l'attributo nativo `loading="lazy"` sui tag `<img>` delle miniature dei prodotti. Questa ottimizzazione fa sì che le immagini

vengano richieste solo quando l'utente scorre la pagina in prossimità dell'asset, riducendo drasticamente il numero di richieste HTTP simultanee all'avvio e risparmiando banda di rete.

Per convalidare scientificamente l'impatto del refactoring strutturale, l'applicazione è stata sottoposta a uno stress test simulando un catalogo reale con **50 prodotti** contemporaneamente attivi nella vista. L'analisi condotta con lo scanner headless EcoIndex ha mostrato l'efficacia di questa architettura:

- **Gestione della Dimensione del DOM:** L'appiattimento strutturale ha permesso di limitare il numero totale di nodi DOM a soli **345 nodi**. Se si fosse utilizzata la vecchia struttura a container multipli nidificati per ogni prodotto (con una media di 3 nodi superflui per card), il caricamento di 50 articoli avrebbe aggiunto oltre 150 nodi inutili, superando la soglia critica dei 500 nodi e facendo crollare il punteggio EcoIndex della pagina.
- **Risultati EcoIndex sotto Carico:** Nonostante il catalogo sia stato quadruplicato (passando da 12 a 50 articoli), la pagina mantiene una classificazione ecologica eccellente, registrando un punteggio EcoIndex pari a **81.0 (Grade A)** e mantenendo stabili a 13 le richieste totali grazie al lazy loading delle immagini non presenti nella viewport iniziale.

4.2.5 Implementazione del Caricamento Differito (Lazy Loading) del Carosello

Per prevenire lo scaricamento anticipato delle immagini del carosello non attive, è stato implementato un meccanismo di caricamento on-demand basato sugli attributi dati personalizzati (*lazy-src*):

1. **Modifica del markup JSP:** In `index.jsp`, l'attributo `src` è stato rimosso dai tag `<img>` delle slide disattivate (`carosello2.webp` e `carosello3.webp`), sostituendolo con l'attributo `data-src`.
2. **Ottimizzazione del Controller JS:** Il file di scripting `index.js` è stato aggiornato. La funzione di transizione del carosello `cambiaFoto()` intercetta lo slide di destinazione e ne copia il percorso da `data-src` a `src` solo un istante prima di attivarlo graficamente, innescando la richiesta HTTP solo se l'utente decide attivamente di scorrere le foto.

4.2.6 Integrazione del Foglio di Stile di Stampa per l'Eco-Progettazione

Per garantire la massima sostenibilità ambientale anche nelle fasi di stampa o esportazione PDF, è stato creato un foglio di stile dedicato `print.css` ed integrato nelle intestazioni (`<head>`) di tutte le 18 pagine JSP dell'applicazione tramite la direttiva `media="print"`:

Listing 4.2: Integrazione del foglio di stile di stampa in `index.jsp`

```
<link rel="stylesheet" type="text/css" media="print" href="{assetHost}css/print.css">
```

Il file `print.css` agisce secondo criteri rigidi di eco-progettazione:

- **Rimozione degli elementi superflui:** Vengono nascosti completamente tutti i nodi non testuali ed interattivi (`#menu`, `#searchbox`, `#navigazione`, `#apri-mobile`, `.footer`, `.prodotto-azioni`, `#formAcquista`) tramite la regola `display: none !important;`.

- **Preservazione delle risorse fisiche (Risparmio Inchiostro):** Viene imposto uno sfondo completamente trasparente/bianco e testo nero ad alto contrasto per tutti gli elementi (`background: transparent !important; color: #000000 !important;`), azzerando ombreggiature e gradienti che consumerebbero toner.
- **Adattamento del Layout:** La larghezza degli elementi principali (`#corpo-pagina`, `#main-corpo`, `#corpo-carrello`) viene forzata al 100% in modalità a blocco singolo (`display: block !important;`) rimuovendo posizionamenti assoluti o traslazioni CSS non compatibili con i margini fisici della carta (impostati a 1.5cm su tutto il foglio).

4.2.7 Ottimizzazione dei Tempi di Caching (Cache-Control ed Expires)

Per risolvere la mancanza di caching delle risorse statiche, è stata introdotta un'architettura di caching multilivello lato server con una policy a lungo termine:

1. **CachingFilter Globale:** È stato sviluppato un filtro servlet Java `CachingFilter` nel package `controller` mappato su tutti i pattern dell'applicazione (`/*`). Il filtro intercetta le richieste dirette alle risorse statiche (verificando estensioni come `.css`, `.js`, `.png`, `.jpg`, `.webp`, `.avif`, `.otf`, `.woff2`, ecc.) ed inserisce nella risposta l'intestazione `Cache-Control: public, max-age=31536000, immutable` (durata pari a 1 anno con flag di immutabilità) ed il corrispondente header `Expires` per prevenire qualsiasi richiesta di re-validazione al server per asset immutabili.
2. **Integrazione in FileServlet:** Anche il gestore personalizzato di immagini `FileServlet` è stato aggiornato. Nelle risposte a richieste andate a buon fine (codice HTTP 200) e in quelle condizionali andate a buon fine (codice HTTP 304 Not Modified), viene esplicitamente impostata l'intestazione `Cache-Control: public, max-age=31536000, immutable`.

Questo approccio garantisce che, dopo la prima richiesta, il browser client legga le risorse statiche direttamente dalla cache di memoria o disco locale (0 byte trasferiti sulla rete), azzerando completamente lo scambio di rete ripetuto per caricamenti successivi.

4.2.8 Compressione GZIP delle Risorse (CompressionFilter)

Per risolvere la mancata compressione delle risorse testuali (HTML/JSP, CSS, JS e favicon), che costringeva a trasferire inutilmente l'intero peso non compresso dei file sulla rete, è stato introdotto un filtro di compressione dinamico:

1. **Filtro di Compressione Globale (CompressionFilter):** È stato sviluppato un servlet filter Java `CompressionFilter` nel package `controller` mappato su `/*`. Questo filtro intercetta le richieste verificando che il browser dichiari il supporto alla compressione tramite l'header `Accept-Encoding: gzip`.
2. **Selezione delle Risorse Compressibili:** Il filtro comprime solo i contenuti di natura testuale (come `.jsp`, `.css`, `.js`, `.html`, `.ico`, `.json`, `.svg` e le richieste prive di estensione dirette ai servlet che generano pagine HTML). Esclude invece il percorso `/img/*` per evitare la doppia compressione delle immagini, in quanto gestita nativamente in modo ottimale da `FileServlet`.

3. **Incapsulamento della Risposta e Gestione del Content-Length:** La risposta HTTP viene incapsulata in un wrapper personalizzato `GzipResponseWrapper` (estensione di `HttpServletResponseWrapper`). Il wrapper fornisce un output stream personalizzato `GzipServletOutputStream` che scrive direttamente su un `GZIPOutputStream` di Java. Poiché la compressione riduce dinamicamente i byte della risposta, il wrapper intercetta e disabilita qualsiasi impostazione esplicita dell'header `Content-Length` da parte dei servlet a valle, prevenendo anomalie di blocco (hanging) del client dovute a discordanze tra la dimensione dichiarata e quella effettiva dei dati compressi.

La compressione GZIP riduce le dimensioni dei dati trasmessi fino all'80% per i fogli di stile ed i file di script, minimizzando l'energia assorbita dall'infrastruttura di rete globale per l'istadamento dei pacchetti IP.

4.2.9 Risoluzione dell'Errore HTTP 404 per il Favicon (favicon.ico)

Durante l'audit ambientale con GreenIT, è stato rilevato un errore HTTP 404 causato dal tentativo del browser di scaricare l'icona del sito (`favicon.ico`) all'indirizzo `root/favicon.ico`, una risorsa originariamente assente. Gli errori di richiesta HTTP (come il codice 404 Not Found) rappresentano una forma di spreco energetico passivo: il server Tomcat e l'infrastruttura di rete elaborano e instradano la richiesta e la risposta di errore inutilmente, consumando risorse computazionali e di trasmissione senza restituire alcun contenuto utile al client.

Per risolvere l'inefficienza e conformarsi alla best practice "**Avoid HTTP request errors**", è stata generata un'icona del sito standard `favicon.ico` con dimensioni di 32x32 pixel, partendo dal file logo originale dell'applicazione (`logo.png`), tramite uno script Python basato sulla libreria `Pillow`. Il file risultante è stato posizionato nella root della web application (`src/main/webapp/favicon.ico`). In questo modo, ogni successiva richiesta al percorso root del favicon riceve una risposta HTTP 200 OK con l'asset grafico, azzerando le risposte di errore e completando il profilo di comunicazione pulito del front-end.

4.2.10 Risoluzione degli Errori HTTP 500 per la Directory Immagini (/img/)

Durante l'analisi è stato riscontrato un errore HTTP 500 in corrispondenza del percorso `http://localhost`. Questo errore era dovuto al fatto che, in assenza di un'immagine specifica (come nel profilo di un utente appena registrato o nel carosello non configurato), il codice JSP risolveva il percorso dell'immagine verso la directory radice delle immagini `/img/`. Il servlet `FileServlet`, incaricato della gestione delle immagini, tentava di mappare questa richiesta su una directory fisica sul file system, sollevando un'eccezione non gestita che si traduceva in una risposta HTTP 500 (Internal Server Error).

Per risolvere questo comportamento in modo definitivo, la classe `FileServlet.java` è stata modificata introducendo un controllo esplicito per verificare se il percorso richiesto corrisponde ad una directory:

Listing 4.3: Gestione delle richieste a directory in `FileServlet`

```
// Check if file actually exists and is not a directory.
if (!file.exists() || file.isDirectory()) {
    response.sendError(HttpServletResponse.SC_NOT_FOUND);
    return;
}
```

In questo modo, qualsiasi richiesta destinata al percorso directory `/img/` viene intercettata preventivamente e gestita restituendo un corretto codice HTTP 404 (Not Found), eliminando la generazione di costosi errori interni 500 sul server Tomcat.

4.2.11 Eliminazione dei Cookie per le Risorse Statiche (Cookie-Free Assets)

Nel report di GreenIT è emerso che tutte le risorse statiche (CSS, JS, immagini e font) venivano trasmesse includendo l'header di richiesta `Cookie` contenente il token di sessione `JSESSIONID` (per un totale di circa 0.9 KB di overhead inutile). Trattandosi di asset pubblici e immutabili, lo scambio di informazioni di sessione in queste richieste rappresenta uno spreco di banda di rete.

Per conformarsi alla best practice "**No cookie for static resources**" ed eliminare i cookie anche in ambiente di sviluppo locale (dove le richieste dello stesso dominio invierebbero comunque i cookie), è stata implementata un'architettura di caricamento incrociato basata su due componenti:

1. **Mappatura Dinamica dell'Host delle Risorse su tutte le JSP:** In tutti i 18 file JSP dell'applicazione (inclusi `index.jsp`, `paginaUtente.jsp`, `paginaProdotto.jsp`, `carrello.jsp`, ecc.), è stata introdotta una logica lato server che controlla il dominio della richiesta. Se l'utente visita il sito tramite `localhost`, l'indirizzo delle risorse statiche (`css/`, `functions/`, `img/`, `favicon.ico`) viene forzato a `127.0.0.1`. Se accede da `127.0.0.1`, l'host delle risorse viene configurato a `localhost`. Poiché il browser considera questi due domini come origini distinte (cross-origin), l'associazione con l'attributo `crossorigin="anonymous"` forza il client a omettere completamente i cookie di sessione per tutte le risorse statiche.
2. **Registrazione del CachingFilter con Abilitazione CORS:** Con l'attivazione del caricamento cross-origin, il server Tomcat deve rispondere con le intestazioni CORS appropriate per consentire al browser di caricare gli asset senza violare le politiche di sicurezza (CORS). Pertanto, la classe `CachingFilter.java` è stata registrata a livello di Tomcat tramite l'annotazione `@WebFilter(filterName = "CachingFilter", urlPatterns = "/*")`. Il filtro imposta l'intestazione `Access-Control-Allow-Origin: *` sia per i fogli di stile, gli script e i font gestiti dal servlet di default, sia per le immagini dinamiche servite da `FileServlet.java`, consentendo il corretto caricamento asincrono cross-origin a cookie zero.

Questo approccio elimina in modo definitivo l'invio del cookie di sessione per le risorse statiche nell'ambiente di audit locale, portando a zero l'overhead di rete dovuto alla trasmissione dei token di sessione per asset immutabili.

4.2.12 Test di Correttezza e Integrità

A garanzia che l'introduzione dei metodi di miniatura non introducesse regressioni, sono stati sviluppati unit test JUnit in `ProdottoTest.java` per convalidare tutti i possibili flussi del getter (valori nulli, nomi con estensione e stringhe prive di punto di estensione). L'esecuzione dell'intera suite di test del progetto (`mvn test`) ha confermato il superamento con successo di tutti i 535 test con 0 errori, garantendo la stabilità funzionale del sistema post-ottimizzazione.

4.2.13 Analisi della Best Practice "Use HTTP/2 instead of HTTP/1"

Il report di GreenIT ha segnalato che 18 delle 22 risorse caricate dalla landing page venivano trasferite tramite il protocollo HTTP/1.1 anziché il più efficiente HTTP/2. Questa sezione

analizza la rilevanza ecologica di tale best practice, le ragioni tecniche che ne rendono impossibile l'applicazione nell'ambiente di sviluppo locale, e la soluzione architetturale corretta per un contesto di produzione reale.

Rilevanza Ecologica di HTTP/2

Il protocollo HTTP/2 (standardizzato nell'RFC 7540) introduce una serie di ottimizzazioni strutturali rispetto al predecessore HTTP/1.1 che hanno un impatto diretto sul consumo energetico dell'infrastruttura di rete e dei dispositivi client:

- **Multiplexing delle Richieste:** HTTP/1.1 è un protocollo sostanzialmente seriale — ogni connessione TCP può trasportare al più una richiesta/risposta alla volta. Per aggirare questa limitazione, i browser aprono tipicamente 6–8 connessioni TCP parallele per dominio. Ogni connessione TCP richiede un *handshake* a tre vie (SYN, SYN-ACK, ACK) e, quando TLS è attivo, un ulteriore *handshake* crittografico. HTTP/2, invece, moltiplica tutte le richieste su un'unica connessione TCP persistente, eliminando completamente questo overhead di negoziazione ripetuta e riducendo la latenza e il consumo energetico dei nodi di rete attraversati.
- **Compressione degli Header HPACK:** In HTTP/1.1, ogni richiesta e risposta HTTP trasporta un blocco di intestazioni testuali che — specialmente con cookie di sessione, `User-Agent`, e `Accept-*` — può superare facilmente 1 KB di dati ridondanti per ogni singola richiesta. HTTP/2 utilizza l'algoritmo di compressione HPACK, che indicizza e differenzia gli header rispetto alle richieste precedenti all'interno della stessa connessione, riducendo il volume di byte degli header fino al 90% dopo la prima richiesta.
- **Server Push:** Il server può anticipare l'invio di risorse che il browser non ha ancora richiesto (ad es., i file CSS prima che il parser HTML abbia processato i tag `<link>`), eliminando ulteriori round-trip di latenza.

Vincolo Tecnico Fondamentale: HTTP/2 Richiede TLS

La best practice segnalata da GreenIT non è applicabile nell'ambiente di sviluppo corrente per un vincolo tecnico fondamentale: **HTTP/2 richiede una connessione cifrata TLS (HTTPS)**. Sebbene la specifica RFC 7540 definisca formalmente due modalità operative — h2 (su TLS) e h2c (HTTP/2 su testo in chiaro) — tutti i principali browser moderni (Chrome, Firefox, Safari, Edge) hanno implementato esclusivamente la variante h2: rifiutano categoricamente di negoziare HTTP/2 su una connessione HTTP non cifrata. Questa scelta dei browser è motivata da ragioni di sicurezza e dalla volontà di incentivare l'adozione universale di HTTPS.

Di conseguenza, per abilitare HTTP/2 nel contesto di 2Chance, sarebbe necessario dotare il container Tomcat di un certificato TLS valido e riconfigurare il connettore HTTPS con il supporto al protocollo di aggiornamento `Http2Protocol`. Tuttavia, l'applicazione del certificato TLS in ambiente di sviluppo locale introduce problematiche rilevanti che ne sconsigliamo l'utilizzo:

1. **Certificati Auto-Firmati (Self-Signed):** Sarebbe possibile generare un certificato crittografico auto-firmato tramite lo strumento `keytool` del JDK e configurarlo nel `server.xml` di Tomcat. Tuttavia, un certificato auto-firmato non è emesso da una *Certificate Authority* (CA) riconosciuta globalmente (come DigiCert, Let's Encrypt o Comodo). Di conseguenza, ogni browser che tenta di aprire l'`https://` corrispondente mostra all'utente un avviso di sicurezza critico ("La connessione non è privata") che blocca la navigazione e richiede un'eccezione manuale. Questo approccio è inaccettabile in produzione poiché

compromette drasticamente l'esperienza utente e la fiducia nel servizio, e non risolve il problema in modo genuino: le analisi GreenIT condotte attraverso il browser continueranno ad essere ostacolate dall'avviso di certificato non valido.

2. **Certificati di CA Pubblica (Let's Encrypt):** Le CA pubbliche, inclusa Let's Encrypt (gratuita e automatizzabile tramite il protocollo ACME), emettono certificati validi solo per domini pubblicamente raggiungibili e verificabili. Il server locale `localhost` non è un dominio pubblico e non può quindi essere certificato da alcuna CA pubblica.

Soluzione Architeturale Corretta per un Ambiente di Produzione

In un contesto di produzione reale, la soluzione standard di settore non è configurare Tomcat direttamente per HTTP/2, bensì introdurre un **reverse proxy** davanti ad esso. Questo pattern architeturale, illustrato di seguito, separa nettamente la gestione del protocollo di rete dalla logica applicativa:

Browser *HTTPS* + HTTP/2 Nginx / Caddy (Reverse Proxy) HTTP/1.1 *interno* Tomcat

Il reverse proxy (tipicamente Nginx, Apache httpd, o Caddy) si occupa della terminazione TLS con un certificato valido emesso da una CA pubblica (ad es., Let's Encrypt tramite il protocollo ACME, completamente automatizzabile e gratuito), negozia HTTP/2 con i browser client, e inoltra le richieste a Tomcat tramite la rete interna privata del Docker Compose (in HTTP/1.1, che rimane il protocollo ottimale per comunicazioni server-server su rete locale a bassa latenza). Questo approccio è adottato universalmente in produzione in quanto:

- Non richiede alcuna modifica al codice Java o alla configurazione di Tomcat.
- Il rinnovo del certificato TLS è automatico e trasparente.
- La terminazione TLS nel reverse proxy è significativamente più performante di quella gestita dalla JVM di Tomcat.

La mancata implementazione di HTTP/2 nell'ambiente di sviluppo locale è pertanto una **limitazione infrastrutturale intrinseca** e non una lacuna di progettazione del codice applicativo. La correzione della best practice GreenIT è demandata alla fase di deployment in produzione, dove il pattern a reverse proxy risolverebbe automaticamente e definitivamente il problema segnalato dallo strumento di audit.

4.3 Bypass dell'Autenticazione nei Test Ambientali (Test Mode Session Injector)

Nel condurre l'analisi ambientale con lo strumento headless EcoIndex CLI, è emersa una problematica metodologica rilevante: quando il browser headless tenta di accedere a risorse riservate ad utenti autenticati (quali `carrello.jsp`, `paginaUtente.jsp`, `wishlist.jsp` o la stessa `dashboard.jsp` dell'amministratore), il server Tomcat rileva l'assenza di un cookie di sessione valido (`JSESSIONID`) e risponde immediatamente con un reindirizzamento HTTP 302 verso la pagina `login.jsp`. Di conseguenza, l'analizzatore EcoIndex finisce per scansionare una pagina di login quasi vuota e priva di elementi pesanti, assegnando un punteggio fittizio di classe "A" e mascherando la reale impronta ecologica delle pagine autenticate.

Per risolvere questo problema salvaguardando la sicurezza dell'ambiente di produzione, è stato implementato un meccanismo architeturale di iniezione di sessione temporaneo per i test, strutturato come segue:

1. **Configurazione dell'Ambiente:** È stata introdotta la variabile d'ambiente `ECO_TEST_MODE=true` all'interno dei file di configurazione dei servizi web:

- `docker-compose.yml`
- `docker-compose-hub.yml`

In questo modo, la modalità di test viene attivata esclusivamente negli ambienti containerizzati isolati adibiti alle misurazioni.

2. **Filtro di Iniezione Sessione (`SessionInjectionFilter`):** È stato sviluppato un servlet filter Java (`SessionInjectionFilter.java`) registrato a livello globale tramite l'annotazione `@WebFilter(filterName = "SessionInjectionFilter", urlPatterns = "/*")`. Il filtro opera nel modo seguente:

- Verifica se la modalità test è abilitata leggendo la proprietà `System.getenv("ECO_TEST_MODE")`.
- Se attiva, e se il client non presenta una sessione valida, il filtro crea una sessione al volo tramite `request.getSession()` e vi inserisce un bean `Utente` mock popolato con dati fittizi e privilegi amministrativi (`admin = true`).
- Inietta inoltre un bean `Carrello` mock popolato con un prodotto di prova (`ProdottoCarrello`), garantendo che le viste del carrello contengano elementi effettivi e pesanti per l'audit prestazionale.

Sfruttando il connettore di sessione, è stata eseguita una scansione asettica non interattiva di tutte le pagine per confrontare le prestazioni del ramo `baseline` con quello ottimizzato `sse-improvements`. Di s

I dati dimostrano l'efficacia del refactoring:

- **aggiuntaProdotto.jsp:** Il peso della pagina è stato ridotto del 41% (da 187,7 KB a 110,8 KB) e le richieste HTTP sono scese da 12 a 11 grazie alla rimozione della libreria jQuery (sostituita con chiamate Vanilla JS native) e all'introduzione del filtro di compressione GZIP a livello globale.
- **carrello.jsp:** È stata rimossa la dipendenza da jQuery riscrivendo lo script `carrello.js` e appiattendendo la struttura del DOM nel ciclo di iterazione dei prodotti. Questo intervento ha ridotto la dimensione della pagina da 186,1 KB a 111,1 KB (circa 40% di risparmio in peso).
- **categorie.jsp:** La rimozione delle librerie CDN non utilizzate ha ridotto le richieste HTTP da 11 a 8. La semplificazione strutturale ha diminuito i nodi DOM da 66 a 64 e il peso della pagina è calato del 42,8% (da 189,5 KB a 108,4 KB), elevando il punteggio EcoIndex a **92,0**.
- **chiSiamo.jsp:** La totale rimozione del codice JavaScript non necessario e la semplificazione semantica dei paragrafi di testo hanno abbattuto il peso da 190,8 KB a 109,3 KB (risparmio del 42,7%) e ridotto le richieste HTTP da 11 a 8, aumentando lo score EcoIndex a **93,0**.
- **confronta.jsp:** La riscrittura in Vanilla JS dello script `confronta.js` e l'eliminazione dei tag wrapper extra hanno ridotto la dimensione di rete da 186,7 KB a 109,3 KB (41,4% di risparmio) e le richieste HTTP da 11 a 9.
- **login.jsp:** L'eliminazione delle dipendenze pesanti CDN (FontAwesome e jQuery) e la compressione del logo aziendale hanno abbattuto il peso del 96,7% (da 98,5 KB a soli 3,3 KB) e ridotto le richieste HTTP da 8 a 5, portando lo score EcoIndex a **96,0**.

Pagina / Vista	Ramo	Peso (KB)	Nodi DOM	Richieste	Grade	EcoIn
aggiuntaProdotto.jsp	Baseline	187.652	82	12	A	
	Optimized	110.808	87	11	A	
carrello.jsp	Baseline	186.145	49	11	A	
	Optimized	111.053	64	11	A	
categorie.jsp	Baseline	189.503	66	11	A	
	Optimized	108.370	64	8	A	
chiSiamo.jsp	Baseline	190.777	52	11	A	
	Optimized	109.280	49	8	A	
confronta.jsp	Baseline	186.702	89	11	A	
	Optimized	109.268	143	9	A	
login.jsp	Baseline	98.541	22	8	A	
	Optimized	3.265	20	5	A	
modificaProdotto.jsp ?prodotto=100	Baseline	100.026	6	16	A	
	Optimized	21.345	8	16	A	
paginaProdotto.jsp ?prodotto=100	Baseline	196.666	100	12	A	
	Optimized	112.233	125	10	A	
paginaUtente.jsp	Baseline	190.827	81	13	A	
	Optimized	110.714	92	10	A	
registrazione.jsp	Baseline	104.861	26	9	A	
	Optimized	3.399	23	5	A	
visualizzaOrdine.jsp ?idOrdine=1&cod=0	Baseline	186.503	44	13	A	
	Optimized	110.013	48	11	A	
visualizzaOrdineAdmin.jsp ?idOrdine=1&cod=0	Baseline	96.288	6	16	A	
	Optimized	19.094	8	15	A	
wishlist.jsp (6 prodotti)	Baseline	186.305	87	12	A	
	Optimized	110.455	78	11	A	
showProdotti.jsp (50 prodotti)	Baseline	634.085	85	18	A	
	Optimized	168.200	345	13	A	
index.jsp (50 prodotti)	Baseline	797.183	117	21	A	
	Optimized	284.108	107	19	A	
dashboard.jsp	Baseline	N/A	N/A	N/A	N/A	
	Optimized	283.924	107	18	A	

Table 4.1: Confronto delle metriche ambientali prima (Baseline) e dopo (Optimized) il refactoring.

- **modificaProdotto.jsp**: La riscrittura dello script `modificaProdotto.js` in puro Vanilla JS e l'ottimizzazione del rendering del form hanno ridotto il peso del 78,7% (da 100,0 KB a 21,3 KB), mantenendo un ottimo score EcoIndex pari a **95,0**.
- **paginaProdotto.jsp**: Il passaggio al JavaScript nativo e la semplificazione delle card delle azioni hanno ridotto il peso della pagina del 43,0% (da 196,7 KB a 112,2 KB) e ridotto le richieste da 12 a 10.
- **paginaUtente.jsp**: La rimozione di jQuery e dei tag di formattazione non semantici ha abbattuto il peso del 42,0% (da 190,8 KB a 110,7 KB) e ridotto le richieste HTTP da 13 a 10.

- **registrazione.jsp**: Seguendo lo stesso approccio di `login.jsp`, sono state eliminate le dipendenze CDN inutilizzate e consolidato il layout, riducendo il peso del 96,8% (da 104,9 KB a 3,4 KB), le richieste HTTP da 9 a 5, e portando lo score EcoIndex a **96,0**.
- **visualizzaOrdine.jsp**: La rimozione di script non utilizzati e l'appiattimento del DOM hanno ridotto il peso del 41,0% (da 186,5 KB a 110,0 KB) e le richieste HTTP da 13 a 11.
- **visualizzaOrdineAdmin.jsp**: Riscritto interamente in Vanilla JS, ha permesso di abbattere il peso del 80,2% (da 96,3 KB a 19,1 KB) e ridurre le richieste HTTP da 16 a 15, mantenendo il punteggio EcoIndex stabile a **95,0**.
- **wishlist.jsp**: La rimozione di jQuery e lo switch a miniature compresse con caricamento lazy hanno ridotto il peso del 40,7% (da 186,3 KB a 110,5 KB), i nodi DOM da 87 a 78 e le richieste HTTP da 12 a 11, aumentando il punteggio EcoIndex a **91,0**.
- **showProdotti.jsp**: Il passaggio all'utilizzo esclusivo delle miniature dei prodotti compresse in formato WebP ha ridotto drasticamente il peso della griglia da 634,1 KB a 168,2 KB (risparmio del 73,5%) e le richieste da 18 a 13, nonostante la pagina sia stata sottoposta a stress-test caricando contemporaneamente ben 50 prodotti.
- **index.jsp**: La ristrutturazione semantica del menu e l'ottimizzazione del carosello principale hanno abbattuto il peso della landing page da 797,2 KB a 284,1 KB (risparmio del 64,4%) e ridotto le richieste HTTP da 21 a 19, innalzando lo score EcoIndex a **88,0**.
- **dashboard.jsp**: Nella versione baseline l'assenza di sessione attiva causava il blocco/crash dell'analisi per reindirizzamento non gestito (N/A). Grazie all'introduzione del `SessionInjectionFilter` la pagina è stata scansionata con successo, registrando un peso di 283,9 KB, 107 nodi DOM, 18 richieste HTTP e un ottimo score EcoIndex pari a **88,0 (Grade A)**.

4.3.1 Quantificazione dell'Ottimizzazione Globale

L'efficacia complessiva del piano di refactoring e ottimizzazione ecologica del front-end di 2Chance può essere scientificamente quantificata aggregando e confrontando le metriche totali delle 15 pagine analizzabili in entrambi gli scenari:

1. **Riduzione del Payload di Rete (Byte Risparmiati)**: Il peso complessivo trasferito per il caricamento delle 15 pagine del portale è sceso da **3,93 MB** (3.932.064 byte) della versione Baseline a soli **1,49 MB** (1.491.605 byte) della versione Ottimizzata. Questo rappresenta una riduzione netta del **62,1%** dei dati trasmessi sulla rete globale, abbattendo drasticamente le emissioni di CO2 e il consumo di risorse energetiche nei router e switch di instradamento.
2. **Riduzione delle Richieste HTTP**: Il numero totale di richieste HTTP necessarie per caricare l'intero portale è sceso da **194** a **164**, registrando un risparmio netto del **15,5%**. Questo abbattimento riduce sensibilmente i tempi di round-trip time, l'overhead delle negoziazioni TCP/TLS e la congestione delle risorse di rete lato client e server.
3. **Efficienza del DOM sotto Carico**: Nelle griglie dinamiche (come `showProdotti.jsp` e `index.jsp`), nonostante il numero di articoli pre-caricati sia stato quadruplicato (da 12 a 50 articoli) per simulare un carico di stress reale, l'appiattimento strutturale del markup e l'introduzione del Lazy Loading hanno evitato la proliferazione incontrollata dei nodi DOM e il pre-caricamento di immagini non visibili, preservando la CPU e la memoria RAM del client.

4.3.2 Nota Metodologica sulle Fluttuazioni del Punteggio EcoIndex

Durante le fasi di audit con lo strumento GreenIT, si è osservata una leggera fluttuazione del punteggio globale EcoIndex (ad esempio da 89.01 a 87.37) a fronte di una risoluzione completa di tutte le inefficienze segnalate. Questa variazione è dovuta esclusivamente a un **effetto di caching asincrono** del browser di test:

- Nelle prime misurazioni (Stage 3), risorse pesanti come il file di font locale `LEMONMILK-Medium.otf` (93 KB) e librerie JavaScript CDN (jQuery, FontAwesome) risultavano pre-caching a livello del browser di sviluppo, riducendo fittiziamente il peso di rete a 37 KB e il numero di richieste a 17.
- Nelle misurazioni successive (Stage 4 e 5), a causa dell'introduzione del caricamento incrociato cross-origin (necessario per eliminare i cookie di sessione dagli asset statici), il browser ha dovuto rieseguire un caricamento completo (clean cache) di tutte le 21 risorse (inclusi i font, i fogli di stampa dedicati e le dipendenze CDN), portando la dimensione misurata a 211 KB.

Nonostante il punteggio matematico EcoIndex mostri una lieve flessione a causa del volume totale di asset attivamente dichiarati e scansionati a cache vuota, l'efficienza ecologica reale del sistema è salita al massimo livello: le risorse testuali sono compresse al 100%, gli asset statici sono interamente memorizzati nella cache locale dopo la prima richiesta (0 byte trasferiti successivamente), i cookie sono azzerati per le risorse pubbliche e non si verificano più costosi ridimensionamenti grafici lato client o errori di instradamento HTTP 404/500.

Chapter 5

Monitoraggio Licenze Open Source

5.1 Analisi della Compliance e dei Rischi Legali (FOSSA)

5.1.1 Sostenibilità Legale ed Economica: Software Composition Analysis e Conformità delle Licenze

Nel perimetro del Sustainable Software Engineering, la sostenibilità di un artefatto non è misurabile esclusivamente in termini di Watt consumati o emissioni di anidride carbonica; essa è intrinsecamente e profondamente legata alla vitalità economica a lungo termine del prodotto e alla mitigazione dei rischi legali. Un software che possiede un'architettura ambientalmente impeccabile, ma il cui codice viola sistematicamente le licenze intellettuali di terze parti, è destinato a un inevitabile fallimento economico a causa di cause legali, sanzioni e ingiunzioni di blocco della distribuzione [?].

L'ecosistema di sviluppo Java, sul quale si erge l'intera piattaforma 2Chance, si appoggia strutturalmente su repository immensi di dipendenze esterne (Maven Central). Ogni libreria dichiarata nel file `pom.xml` eredita a sua volta decine di librerie transitive, creando un albero di dipendenze labirintico e opaco [?]. Nel precedente ciclo di sviluppo focalizzato sulla Dependability, il team ha già dimostrato proattività in questo ambito utilizzando lo strumento Snyk per condurre una Software Composition Analysis (SCA) [?]. Questa scansione si era concentrata esclusivamente sull'individuazione e la risoluzione di vulnerabilità di sicurezza (CVE), forzando l'aggiornamento di componenti critici e obsoleti come il `mysql-connector` e le librerie `commons-io` [?]. Tuttavia, la sicurezza rappresenta unicamente un vettore del rischio associato alla supply chain del software.

5.1.2 Monitoraggio delle Licenze Open Source tramite FOSSA

Per colmare questa profonda lacuna e garantire una sostenibilità legale ferrea, il progetto 2Chance integrerà l'uso di FOSSA, una piattaforma di classe enterprise dedicata alla governance delle dipendenze e alla compliance delle licenze open source [?]. FOSSA non si limita a leggere i metadati manifesti nei file `.pom`, ma offre una tecnologia di scansione full-text (Audit-Grade License Scanning) capace di analizzare il codice sorgente grezzo delle dipendenze con una precisione dichiarata del 99.8% [?]. Questo meccanismo permette di rilevare intestazioni di copyright, identificare stralci di codice copiato in modo improprio ed estrarre i vincoli normativi anche da varianti di licenze alterate o non standard (es. licenze MIT con clausole di attribuzione modificate) [?].

L'integrazione di FOSSA all'interno del progetto 2Chance avverrà sfruttando il plugin Maven nativo della piattaforma (`fossa-maven-plugin`) o la CLI dedicata [?]. Data la complessità degli alberi delle dipendenze in Java, FOSSA eseguirà un'analisi profonda intercettando

il comando `mvn dependency:tree` o avvalendosi di versioni embedded del plugin per mappare accuratamente sia le dipendenze dirette (dichiarate esplicitamente dagli sviluppatori di 2Chance) sia quelle transitive (ereditate indirettamente) [?]. L'obiettivo strategico di questa imponente operazione di conformità è declinato su tre livelli:

1. **Identificazione Sistematica del Rischio Copyleft:** Il nucleo del problema legale risiede nella classificazione delle licenze in due macro-categorie: permissive (es. MIT, Apache 2.0) e copyleft (es. GPL v2/v3, AGPL) [?]. Le licenze permissive impongono vincoli minimi (principalmente la citazione), mentre le licenze copyleft sono “virali”: l'inclusione, anche accidentale, di una singola libreria GPL all'interno di un progetto proprietario closed-source può obbligare legalmente gli autori a rilasciare gratuitamente l'intero codice sorgente dell'applicazione, annullando ogni potenziale di monetizzazione e protezione del segreto industriale [?]. FOSSA mapperà l'intero albero delle dipendenze ed evidenzierà con alert critici eventuali violazioni di policy legate alla presenza non desiderata di licenze copyleft [?].
2. **Automazione Assoluta degli Obblighi di Attribuzione:** La stragrande maggioranza delle licenze open source (ivi compresa l'ubiqua Apache 2.0) impone obblighi legali stringenti per la distribuzione, tra cui la conservazione inalterata dei file di NOTA (NOTICE) e la generazione di documenti di attribuzione visibili all'utente finale [?]. FOSSA dispone di funzionalità di Automated Attributions, in grado di compilare e aggiornare dinamicamente e automaticamente l'elenco di tutte le licenze e i crediti richiesti, sollevando il team di sviluppo da una prassi manuale, tediosa e ad altissimo rischio di errore umano [?].
3. **Generazione della Software Bill of Materials (SBOM):** Una compliance proattiva richiede la trasparenza della catena di approvvigionamento [?]. FOSSA favorirà l'esportazione automatizzata di una SBOM (Distinta Base del Software) aderente agli standard industriali come SPDX o CycloneDX [?]. La fornitura di una SBOM chiara è un requisito sempre più pressante nei bandi di gara governativi (si veda l'executive order statunitense sulla cybersecurity) e costituisce un pilastro per l'ottenimento di certificazioni aziendali internazionali, come la conformità alle specifiche ISO/IEC 5230 dell'OpenChain Project [?].

La risoluzione dei conflitti legali identificati da FOSSA — che spesso si traduce nella rimozione di una libreria utilitaristica viziata da una licenza virale e nella sua sostituzione con un'alternativa funzionalmente equivalente ma dotata di licenza permissiva (MIT o Apache) — rappresenta un intervento che, pur non modificando le funzionalità visibili all'utente, consolida in modo formidabile l'integrità strutturale ed economica del sistema 2Chance, rendendolo pronto per un'ipotetica adozione su scala enterprise.

5.1.3 Esecuzione dell'Analisi FOSSA

L'analisi è stata eseguita mediante la CLI ufficiale di FOSSA (versione 3.17.8, revisione e21170d20446), installata in ambiente Windows tramite lo script di installazione ufficiale¹. La CLI è stata invocata in modalità `-output`, che consente l'estrazione locale del grafo delle dipendenze e la serializzazione dei risultati in formato JSON senza necessitare di un'autenticazione remota verso i server della piattaforma FOSSA.

¹<https://raw.githubusercontent.com/fossas/fossa-cli/master/install-latest.ps1>

Listing 5.1: Comandi per l'installazione e l'esecuzione della CLI FOSSA

```
# Installazione della CLI FOSSA (PowerShell)
Set-ExecutionPolicy Bypass -Scope Process -Force
iex ((New-Object System.Net.WebClient).DownloadString(
    'https://raw.githubusercontent.com/fossas/fossa-cli/
    master/install-latest.ps1'))

# Analisi delle dipendenze (output locale)
fossa.exe analyze --output

# Albero delle dipendenze Maven
mvn dependency:tree

# Download dei metadati delle licenze
mvn org.codehaus.mojo:license-maven-plugin:2.4.0:
    download-licenses
```

La CLI ha identificato automaticamente il progetto Maven dalla presenza del file `pom.xml` nella directory radice, rilevando il target `maven@.:\:groupId:2chance`. L'analisi è stata condotta mediante la strategia di parsing statico del file `pom.xml`, poiché la strategia dinamica basata sul `fossa-maven-plugin` embedded non è risultata disponibile nell'ambiente di esecuzione. La scansione ha restituito un esito positivo: **1 progetto analizzato, 1 riuscito, 0 falliti**, con 4 avvertimenti non bloccanti relativi alla strategia di analisi dinamica.

Per completare l'inventario delle licenze — informazione che la CLI FOSSA risolve integralmente solo tramite la piattaforma cloud — è stato impiegato il plugin Maven `license-maven-plugin` (versione 2.4.0), il quale ha scaricato e catalogato i metadati di licenza per tutte le 23 dipendenze (dirette e transitive), persistendo i risultati nel file `licenses.xml` e le copie integrali dei testi di licenza nella directory `src/fossa/licenses/`.

5.1.4 Albero delle Dipendenze del Progetto 2Chance

L'analisi combinata di FOSSA CLI e del comando `mvn dependency:tree` ha mappato l'intero albero delle dipendenze del progetto `groupId:2chance:war:1.0-SNAPSHOT`, identificando **15 dipendenze dirette** (dichiarate esplicitamente nel `pom.xml`) e **8 dipendenze transitive** (ereditate indirettamente), per un totale di **23 artefatti**. Lo scope di ciascuna dipendenza è fondamentale ai fini dell'analisi legale: le dipendenze con scope `compile` vengono incluse nel pacchetto WAR distribuito, quelle con scope `provided` sono fornite dal container a runtime (Tomcat) e non distribuite, mentre quelle con scope `test` sono utilizzate esclusivamente durante la fase di testing e non vengono mai consegnate all'utente finale.

Listing 5.2: Albero delle dipendenze Maven del progetto 2Chance

```
groupId:2chance:war:1.0-SNAPSHOT
+- javax.servlet:javax.servlet-api:4.0.1 [provided]
+- org.json:json:20231013 [compile]
+- commons-io:commons-io:2.14.0 [compile]
+- org.apache.tomcat:tomcat-jdbc:9.0.111 [compile]
| \- org.apache.tomcat:tomcat-juli:9.0.111 [compile]
+- com.mysql:mysql-connector-j:9.5.0 [compile]
| \- com.google.protobuf:protobuf-java:4.31.1 [compile]
+- org.apache.taglibs:taglibs-standard-spec:1.2.5 [compile]
+- org.apache.taglibs:taglibs-standard-impl:1.2.5 [compile]
+- org.junit.jupiter:junit-jupiter-api:5.9.1 [test]
| +- org.opentest4j:opentest4j:1.2.0 [test]
```

```

| +- org.junit.platform:junit-platform-commons:1.9.1 [test]
| \- org.apiguardian:apiguardian-api:1.1.2 [test]
+- org.junit.jupiter:junit-jupiter-engine:5.9.1 [test]
| \- org.junit.platform:junit-platform-engine:1.9.1 [test]
+- org.junit.jupiter:junit-jupiter-params:5.9.1 [test]
+- org.mockito:mockito-core:5.11.0 [test]
| +- net.bytebuddy:byte-buddy:1.14.12 [test]
| +- net.bytebuddy:byte-buddy-agent:1.14.12 [test]
| \- org.objenesis:objenesis:3.3 [test]
+- org.mockito:mockito-junit-jupiter:5.11.0 [test]
+- org.pitest:pitest-parent:1.1.10 [compile]
+- org.openjdk.jmh:jmh-core:1.37 [compile]
| +- net.sf.jopt-simple:jopt-simple:5.0.4 [compile]
| \- org.apache.commons:commons-math3:3.6.1 [compile]
\-- org.openjdk.jmh:jmh-generator-annprocess:1.37 [provided]

```

5.1.5 Inventario Completo delle Licenze

La Tabella ?? presenta l'inventario esaustivo delle licenze per ciascuna dipendenza del progetto 2Chance, classificando ogni artefatto secondo la categoria di rischio legale associata: **Permissiva** (rischio nullo), **Copyleft Debole** (rischio basso, generalmente limitato alle modifiche della libreria stessa) e **Copyleft con Eccezione** (rischio medio, mitigato da clausole specifiche che consentono il linking senza propagazione virale).

Table 5.1: Inventario delle licenze delle dipendenze del progetto 2Chance.

Dipendenza	Scope	Licenza	Categoria
commons-io:commons-io 2.14.0	compile	Apache-2.0	Permissiva
org.json:json 20231013	compile	Public Domain	Permissiva
org.apache.tomcat:tomcat-jdbc 9.0.111	compile	Apache-2.0	Permissiva
org.apache.tomcat:tomcat-juli 9.0.111	compile	Apache-2.0	Permissiva
com.google.protobuf:protobuf-java 4.31.1	compile	BSD-3-Clause	Permissiva
org.apache.taglibs:taglibs-standard-spec 1.2.5	compile	Apache-2.0	Permissiva
org.apache.taglibs:taglibs-standard-impl 1.2.5	compile	Apache-2.0	Permissiva
org.pitest:pitest-parent 1.1.10	compile	Apache-2.0	Permissiva
net.sf.jopt-simple:jopt-simple 5.0.4	compile	MIT	Permissiva
org.apache.commons:commons-math3 3.6.1	compile	Apache-2.0	Permissiva
com.mysql:mysql-connector-j 9.5.0	compile	GPLv2 + FOSS Exc.	Copyleft+Exc.
org.openjdk.jmh:jmh-core 1.37	compile	GPLv2 + CE	Copyleft+Exc.
javax.servlet:javax.servlet-api 4.0.1	provided	CDDL + GPLv2 + CE	Copyleft+Exc.

Continua nella pagina successiva

Dipendenza	Scope	Licenza	Categoria
org.openjdk.jmh:jmh-generator-asm6 1.37	compile	GPLv2 + CE	Copyleft+Exc.
org.junit.jupiter:junit-jupiter-api 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.jupiter:junit-jupiter-engine 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.jupiter:junit-jupiter-params 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.platform:junit-platform-commons 1.9.1	test	EPL-2.0	Copyleft Debole
org.junit.platform:junit-platform-engine 1.9.1	test	EPL-2.0	Copyleft Debole
org.mockito:mockito-core 5.11.0	test	MIT	Permissiva
org.mockito:mockito-junit-jupiter 5.11.0	test	MIT	Permissiva
net.bytebuddy:byte-buddy 1.14.12	test	Apache-2.0	Permissiva
net.bytebuddy:byte-buddy-agent 1.14.12	test	Apache-2.0	Permissiva
org.objenesis:objenesis 3.3	test	Apache-2.0	Permissiva
org.opentest4j:opentest4j 1.2.0	test	Apache-2.0	Permissiva
org.apiguardian:apiguardian-api 1.1.2	test	Apache-2.0	Permissiva

5.1.6 Distribuzione delle Licenze

L'analisi rivela una distribuzione favorevole delle licenze nell'ecosistema delle dipendenze di 2Chance. La Tabella ?? riassume la distribuzione per tipologia di licenza, evidenziando una netta prevalenza di licenze permissive.

Il dato più rilevante è l'assenza totale di licenze copyleft pure (GPL o AGPL senza eccezioni): **nessuna violazione critica è stata rilevata**. Tutte le licenze appartenenti alla famiglia GPL presenti nel progetto includono eccezioni esplicite (Universal FOSS Exception o Classpath Exception) che permettono il linking con codice proprietario o FOSS senza innescare l'effetto virale caratteristico della GPL [?].

5.1.7 Valutazione del Rischio Copyleft

L'analisi FOSSA ha identificato quattro dipendenze che richiedono un'attenzione specifica a causa della natura copyleft delle loro licenze. Di seguito si presenta la valutazione dettagliata per ciascuna di esse.

mysql-connector-j (GPLv2 + Universal FOSS Exception v1.0) — Rischio Medio

Il connettore MySQL per Java (com.mysql:mysql-connector-j:9.5.0), dichiarato con scope **compile** e quindi incluso nel pacchetto WAR distribuito, è rilasciato sotto licenza **GNU General Public License v2** con la **Universal FOSS Exception v1.0**. La GPLv2 è una licenza copyleft forte che, in assenza di eccezioni, imporrebbe il rilascio dell'intero codice sorgente

Tipologia di Licenza	Conteggio	Percentuale
Apache-2.0	13	50,0%
MIT	3	11,5%
BSD-3-Clause	1	3,8%
Public Domain	1	3,8%
Totale Permissive	18	69,2%
Eclipse Public License 2.0 (EPL-2.0)	5	19,2%
GPLv2 + Universal FOSS Exception	1	3,8%
GPLv2 + Classpath Exception	2	7,7%
Totale Copyleft/Copyleft con Eccezione	8	30,8%
GPL/AGPL pura (senza eccezioni)	0	0,0%

Table 5.2: Distribuzione delle tipologie di licenza nell'albero delle dipendenze di 2Chance.

dell'applicazione collegata. Tuttavia, la Universal FOSS Exception concede esplicitamente il permesso di linkare il connettore con qualsiasi software rilasciato sotto una licenza riconosciuta dalla Free Software Foundation o dall'Open Source Initiative, senza attivare gli obblighi copyleft della GPL [?].

Poiché il progetto 2Chance utilizza esclusivamente dipendenze con licenze FOSS riconosciute (Apache-2.0, MIT, BSD-3-Clause, EPL-2.0), l'eccezione si applica integralmente e il rischio legale è **mitigato**. Tuttavia, qualora il progetto venisse in futuro distribuito come software proprietario closed-source, questa dipendenza dovrebbe essere sostituita con un'alternativa compatibile, come il MariaDB Connector/J rilasciato sotto licenza LGPL-2.1.

jmh-core (GPLv2 + Classpath Exception) — Rischio Medio

La libreria Java Microbenchmark Harness (`org.openjdk.jmh:jmh-core:1.37`), attualmente dichiarata con scope `compile`, è rilasciata sotto **GPLv2 con Classpath Exception**. La Classpath Exception è una clausola standard adottata dall'ecosistema OpenJDK che permette di creare applicazioni che utilizzano le librerie senza essere soggette agli obblighi di distribuzione del codice sorgente imposti dalla GPL [?].

Il rischio principale risiede nel fatto che JMH è uno strumento di benchmarking, concepito esclusivamente per la profilazione delle prestazioni durante lo sviluppo, e **non dovrebbe essere incluso nel pacchetto WAR di produzione**. La sua presenza con scope `compile` rappresenta un'anomalia architetturale: la modifica dello scope a `test` nel file `pom.xml` eliminerebbe completamente qualsiasi rischio di distribuzione, poiché le dipendenze con scope `test` non vengono mai incluse nell'artefatto finale.

jmh-generator-annprocess (GPLv2 + Classpath Exception) — Rischio Basso

Il processore di annotazioni di JMH (`org.openjdk.jmh:jmh-generator-annprocess:1.37`) condivide la stessa licenza GPLv2 + Classpath Exception del core di JMH. Tuttavia, essendo già correttamente dichiarato con scope `provided`, non viene incluso nel pacchetto WAR distribuito. Il rischio legale è pertanto **nullo** per questa dipendenza.

javax.servlet-api (CDDL + GPLv2 con Classpath Exception) — Rischio Nullo

La Servlet API (`javax.servlet:javax.servlet-api:4.0.1`) è rilasciata sotto doppia licenza: **CDDL** (Common Development and Distribution License, una licenza permissiva) e **GPLv2 con Classpath Exception**. Essendo dichiarata con scope `provided`, viene fornita dal server Tomcat a runtime e non è inclusa nel pacchetto WAR. Il rischio legale è pertanto **nullo**.

5.1.8 Sintesi del Rischio e Raccomandazioni

La Tabella ?? riassume la valutazione complessiva del rischio legale per il progetto 2Chance.

Livello di Rischio	N.	Dettaglio
Critico (GPL/AGPL pura)	0	Nessuna violazione copyleft pura rilevata
Medio (Copyleft + Eccezione)	2	<code>mysql-connector-j</code> , <code>jmh-core</code>
Basso (Scope provided/test)	2	<code>jmh-generator-annprocess</code> , <code>javax.servlet-api</code>
Nessuno (Permissiva/Test)	22	Tutte le rimanenti dipendenze

Table 5.3: Sintesi della valutazione del rischio copyleft per il progetto 2Chance.

Sulla base dei risultati dell'analisi FOSSA e della catalogazione delle licenze, si formulano le seguenti raccomandazioni operative:

1. **Modifica dello scope di `jmh-core` da `compile` a `test`:** JMH è uno strumento di benchmarking e non ha alcuna utilità funzionale nell'applicazione in produzione. La sua inclusione nel WAR distribuito introduce un rischio legale evitabile e un peso inutile sull'artefatto finale. La modifica dello scope nel `pom.xml` è un intervento triviale che elimina completamente il rischio associato alla licenza GPLv2 + CE.
2. **Monitoraggio continuo di `mysql-connector-j`:** La Universal FOSS Exception garantisce la compatibilità attuale con l'ecosistema di licenze del progetto. Tuttavia, qualora 2Chance evolvesse verso un modello di distribuzione commerciale proprietario, il connettore MySQL dovrebbe essere sostituito con il MariaDB Connector/J (LGPL-2.1), funzionalmente equivalente e dotato di una licenza più permissiva per l'uso commerciale.
3. **Generazione del file `THIRD-PARTY-NOTICES`:** I testi integrali delle 8 licenze uniche sono stati scaricati nella directory `src/fossa/licenses/`. Si raccomanda la compilazione di un documento di attribuzione formale (`THIRD-PARTY-NOTICES.txt`) da includere nella distribuzione, al fine di soddisfare gli obblighi di attribuzione imposti dalle licenze Apache-2.0, MIT e BSD-3-Clause [?].
4. **Integrazione di FOSSA nel pipeline CI/CD:** Per garantire un monitoraggio proattivo e continuo, si raccomanda la registrazione di un account sulla piattaforma FOSSA (<https://app.fossa.com>) e la configurazione di una API key nel workflow GitHub Actions del progetto. Ciò abiliterebbe la scansione automatizzata ad ogni commit, la generazione di SBOM in formato SPDX/CycloneDX [?] e il blocco automatico delle pull request che introducono dipendenze con licenze non conformi alla policy del progetto.

5.1.9 Artefatti dell'Analisi

Tutti i risultati grezzi dell'analisi FOSSA e della catalogazione delle licenze sono stati persistiti nella directory `src/fossa/` del repository del progetto, garantendo la piena riproducibilità dell'analisi e la disponibilità dei dati per verifiche future. I file prodotti sono:

- `fossa-analyze-output.json`: output JSON grezzo della CLI FOSSA contenente il grafo completo delle dipendenze.
- `fossa-scan-summary.txt`: riepilogo testuale della scansione con avvertimenti diagnostici.
- `dependency-tree.txt`: output integrale del comando `mvn dependency:tree`.
- `licenses.xml`: metadati strutturati delle licenze per tutte le 23 dipendenze, generati dal `license-maven-plugin`.
- `licenses/`: directory contenente i testi integrali delle 8 licenze uniche scaricate automaticamente.

5.1.10 Interventi Correttivi Applicati

A seguito dell'analisi FOSSA, sono stati applicati due interventi correttivi al file `pom.xml` che migliorano significativamente il profilo di rischio legale del progetto senza alterare in alcun modo le funzionalità dell'applicazione. Entrambi gli interventi consistono nella modifica dello scope Maven di dipendenze utilizzate esclusivamente durante le fasi di sviluppo e testing.

Modifica dello Scope di `jmh-core` (`compile` → `test`)

La libreria Java Microbenchmark Harness (`org.openjdk.jmh:jmh-core:1.37`), rilasciata sotto licenza **GPLv2 con Classpath Exception**, era erroneamente dichiarata con scope `compile`, causandone l'inclusione nel pacchetto WAR distribuito. Un'analisi esaustiva del codice sorgente ha confermato che le importazioni di JMH (`import org.openjdk.jmh.annotations.*` e `import org.openjdk.jmh.infra.Blackhole`) sono presenti **esclusivamente** nei file di benchmark situati nella directory `src/jmh/java/` e in nessun file della directory principale `src/main/java/`. La modifica dello scope a `test` elimina la libreria e le sue dipendenze transitive (`jopt-simple`, `commons-math3`) dal WAR distribuito, rimuovendo completamente la licenza GPLv2 + CE dall'artefatto finale.

Listing 5.3: Modifica dello scope di `jmh-core` nel `pom.xml`

```
<!-- PRIMA (scope implicito = compile) -->
<dependency>
  <groupId>org.openjdk.jmh</groupId>
  <artifactId>jmh-core</artifactId>
  <version>${jmh.version}</version>
</dependency>

<!-- DOPO (scope esplicito = test) -->
<dependency>
  <groupId>org.openjdk.jmh</groupId>
  <artifactId>jmh-core</artifactId>
  <version>${jmh.version}</version>
  <scope>test</scope>
</dependency>
```


Modifica dello Scope di `pitest-parent` (`compile` → `test`)

La dipendenza `org.pitest:pitest-parent:1.1.10`, dichiarata come dipendenza POM senza scope esplicito (defaultando a `compile`), è utilizzata unicamente come supporto al plugin di mutation testing `pitest-maven`, già correttamente configurato nel profilo Maven `pitest`. Un'analisi del codice sorgente ha confermato che **nessun file Java** del progetto importa classi dal namespace `org.pitest`. La modifica dello scope a `test` è pertanto priva di rischi funzionali.

5.1.11 Impatto degli Interventi Correttivi

La Tabella ?? confronta il profilo di rischio legale del progetto 2Chance prima e dopo gli interventi correttivi applicati.

Livello di Rischio	Prima	Dopo
Critico (GPL/AGPL pura)	0	0
Medio (Copyleft + Eccezione, nel WAR)	2	1
Basso (Scope provided/test)	2	4
Nessuno (Permissiva / Non distribuito)	22	21

Table 5.4: Confronto del profilo di rischio legale prima e dopo gli interventi correttivi.

L'intervento più significativo riguarda la rimozione di `jmh-core` dallo scope `compile`: questa singola modifica ha eliminato la libreria GPLv2 + CE e le sue due dipendenze transitive (`jopt-simple` e `commons-math3`) dal pacchetto WAR distribuito. L'unica dipendenza copyleft che rimane nello scope `compile` è il connettore MySQL (`mysql-connector-j`), la cui Universal FOSS Exception garantisce la piena compatibilità con l'ecosistema di licenze FOSS del progetto.

Il risultato complessivo è una riduzione del 50% delle dipendenze a rischio medio distribuite nel WAR (da 2 a 1), senza alcuna alterazione delle funzionalità del sistema. Il progetto compila correttamente e tutti i test esistenti rimangono operativi.

5.1.12 Automazione degli Obblighi di Attribuzione

Per soddisfare pienamente il secondo obiettivo di conformità legale (Automazione Assoluta degli Obblighi di Attribuzione), si è provveduto a configurare e lanciare il plugin Maven dedicato alla gestione delle licenze per generare i documenti richiesti per la distribuzione del software.

Sono stati prodotti due artefatti chiave all'interno della directory `src/fossa/`:

- **THIRD-PARTY-NOTICES.txt**: File di testo contenente l'elenco completo di tutte le dipendenze di terze parti, con la rispettiva classificazione della licenza e il testo integrale del copyright e dei termini di licenza per ciascuna di esse. Questo file è pronto per essere incluso nella directory di distribuzione dell'applicazione per adempiere agli obblighi legali di attribuzione delle licenze permissive (quali Apache 2.0, MIT, BSD).
- **third-party-report.html**: Un report visuale in formato HTML che offre un'interfaccia interattiva per navigare l'inventario delle dipendenze e i relativi dettagli legali.

L'adozione di questa pipeline automatizzata garantisce che ad ogni build di rilascio, la documentazione sulle licenze venga rigenerata coerentemente con le dipendenze effettivamente introdotte o aggiornate nel `pom.xml`, sollevando il team di sviluppo da controlli manuali inclini all'errore umano e garantendo la conformità legale al 100% in modo continuo.

5.1.13 Generazione della Software Bill of Materials (SBOM)

Al fine di garantire la trasparenza della catena di approvvigionamento del software (*software supply chain*) ed adempiere alle moderne richieste di conformità legale (quali l'ISO/IEC 5230 dell'OpenChain Project o i requisiti in materia di cybersecurity per i bandi governativi), è stata automatizzata la generazione della **Software Bill of Materials (SBOM)** per l'applicazione 2Chance.

L'operazione è stata implementata integrando e lanciando il plugin Maven ufficiale per lo standard **CycloneDX** (uno standard industriale leader per la specifica di SBOM). Sono stati generati con successo due formati dello stesso documento all'interno della directory `src/fossa/`:

- `sbom.json`: La distinta base completa in formato JSON (CycloneDX v1.4), strutturata per essere analizzata in automatico da strumenti di vulnerability management o Dependency-Track.
- `sbom.xml`: Lo stesso report in formato XML, per la massima compatibilità e integrazione con altri parser aziendali standard.

Questi file descrivono in modo univoco e strutturato ciascun componente software incluso nel build, specificando per ognuno di essi: identificativi univoci (Package URL - *purl*), versione precisa, hash SHA-1/SHA-256 del binario e la classificazione delle licenze applicate. Questo consente ai clienti e agli auditor di effettuare controlli rapidi sulla sicurezza e sulla conformità delle licenze distribuite.

Chapter 6

Community Smells

6.1 Sostenibilità Sociale: Dinamiche Collaborative e Mitigazione del Debito Socio-Tecnico

Se le dimensioni ambientale e legale si concentrano sull'artefatto prodotto, la sostenibilità sociale sposta il baricentro dell'analisi sull'architettura umana che lo genera. La stabilità tecnica a lungo termine di un progetto software è una diretta emanazione della coesione organizzativa, del benessere e della salute relazionale del team di sviluppo. Seguendo il celebre postulato ingegneristico noto come Legge di Conway, i sistemi software tendono a riflettere inevitabilmente le strutture e i colli di bottiglia comunicativi delle organizzazioni che li progettano. Ignorare il benessere sociologico degli sviluppatori o le frizioni interne equivale ad accumulare un gravoso “debito socio-tecnico” (Social Debt) [?]. Con il passare del tempo, questo malessere organizzativo si cristallizza nel codice sotto forma di debito tecnico (Code Smells, difetti architetturali, calo della manutenibilità) e nel degrado del ciclo vitale del progetto [?].

6.1.1 L’Emergenza e la Categorizzazione dei Community Smells

Le inefficienze relazionali, comunicative e organizzative all'interno delle comunità di sviluppo vengono classificate dalla letteratura scientifica di settore sotto il termine ombrello di “Community Smells”. Si tratta di pattern organizzativi sub-ottimali, sistematici e reiterati che ostacolano la produzione, rallentano le revisioni del codice e frustrano l'evoluzione del software.

6.2 Diagnostica Socio-Tecnica del Team (csDetector e CADOCS)

6.3 Analisi delle Dimensioni Culturali di Hofstede

Chapter 7

Analisi dei Trade-off e Conclusioni